

Вы читаете бесплатно по [ссылке](#) от Фариды Хана. [Станьте участником](#) , чтобы получить доступ к лучшим материалам на Medium.

★ История только для участников

ИИ



Искусственный Интеллект



Наука о данных



Машинное обучение



Питон



Создание быстрых компьютерных агентов для решения сложных задач

Аргументы в пользу переноса интерфейса в ИИ-агенты

37 минут чтения · 1 час назад



Фарид Хан

Подписаться



Слушай

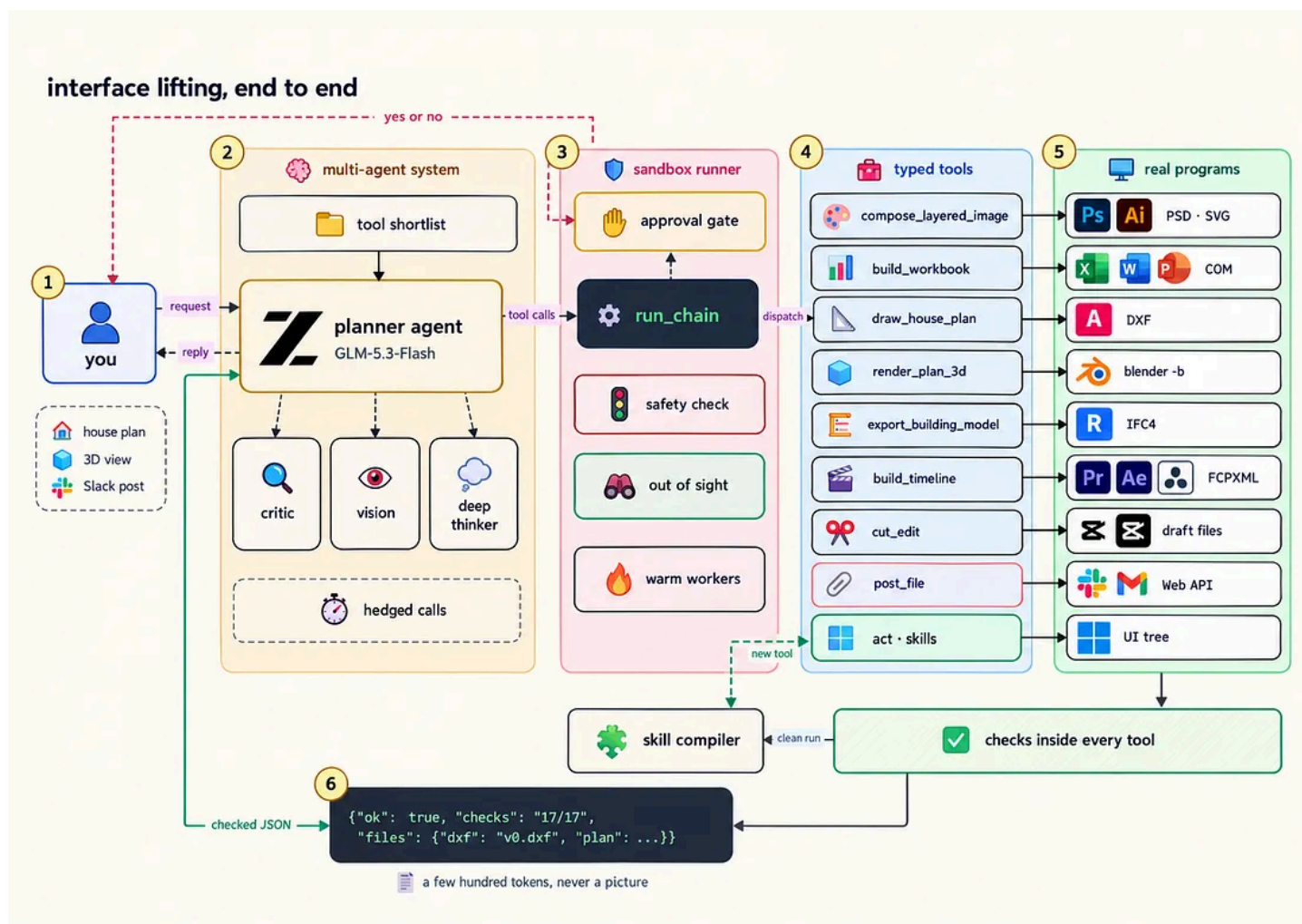


Поделиться

Прочитайте эту статью бесплатно: [ссылка](#)

Современные компьютерные агенты, такие как [Computer-Using Agent](#) от OpenAI и [computer use](#) от Anthropic, работают со скриншотами. Каждый клик требует вызова модели для обработки большого изображения, а при выполнении сложных и длительных задач эти вызовы быстро накапливаются — как по времени, так и по количеству токенов. Когда задача возвращается, агент снова изучает всю программу. Современный подход к разработке программного обеспечения под названием **interface lifting** делает само программное обеспечение доступным для агента. Каждая функция становится типизированным инструментом, который управляет программой через

собственный файл, командную строку или API. Модель по-прежнему принимает все решения, но за несколько вызовов вместо десятков кликов.



Комплексный интерфейс: запрос проходит через команду агентов, песочницу и специализированные инструменты, которые обращаются к реальным программам, и возвращается в виде проверенного результата в формате JSON (создано [Фаридом Ханом](#))

Вот архитектура, которую мы будем создавать, по одному компоненту за раз:

- **Мультиагентная система:** агент-планировщик пишет вызовы инструментов и целые цепочки, а рядом с ним работают критик, агент визуального восприятия и генератор идей, а также формируется шорт-лист, в который для каждого запроса попадают только релевантные инструменты.
- **Песочница:** вызовы инструментов выполняются за одобрительным шлюзом для всего внешнего и классификатором безопасности для каждого действия в пользовательском интерфейсе. Программы работают без графического интерфейса, вне экрана и в режиме ожидания.

- **Типизированный реестр инструментов:** обычные функции Python становятся инструментами, каждый из которых имеет схему, которую считывает модель, реализацию, верификатор и эффект.
- **Нативные интерфейсы до появления пикселей:** каждый инструмент запускает свою программу через собственный файл, командную строку, СОМ-интерфейс или API, и только потом подключается дерево доступности.
- **Проверенные результаты в виде текста:** каждый инструмент измеряет то, что он сделал, и отправляет обратно несколько сотен токенов JSON.
- **Компилятор навыков:** задача, которая когда-то выполнялась, становится новым инструментом, который может вызывать модель.
- **Реестр и исследование с удалением:** каждая задача сравнивается с экранным агентом, а каждый дизайнерский выбор оценивается.

К концу этого поста наш агент нарисует план дома в AutoCAD за 4,4 секунды, сделав 2 вызова модели, свяжет AutoCAD, Blender и Slack в одно предложение с помощью одного вызова планирования и превратит изученную задачу для настольного компьютера в новый инструмент, который воспроизводит действия за 56–64 миллисекунды.

Весь код доступен в моем репозитории на GitHub: [репозиторий GitHub для быстрого компьютерного агента](#). Кодовая база организована следующим образом.

GitHub — FareedKhan-dev/ai-pc: помощник для Windows, который работает в 118 настольных программах и...

Помощник для Windows, который работает со 118 настольными программами и сервисами, используя код вместо скриншотов....

github.com

```
agent/
```

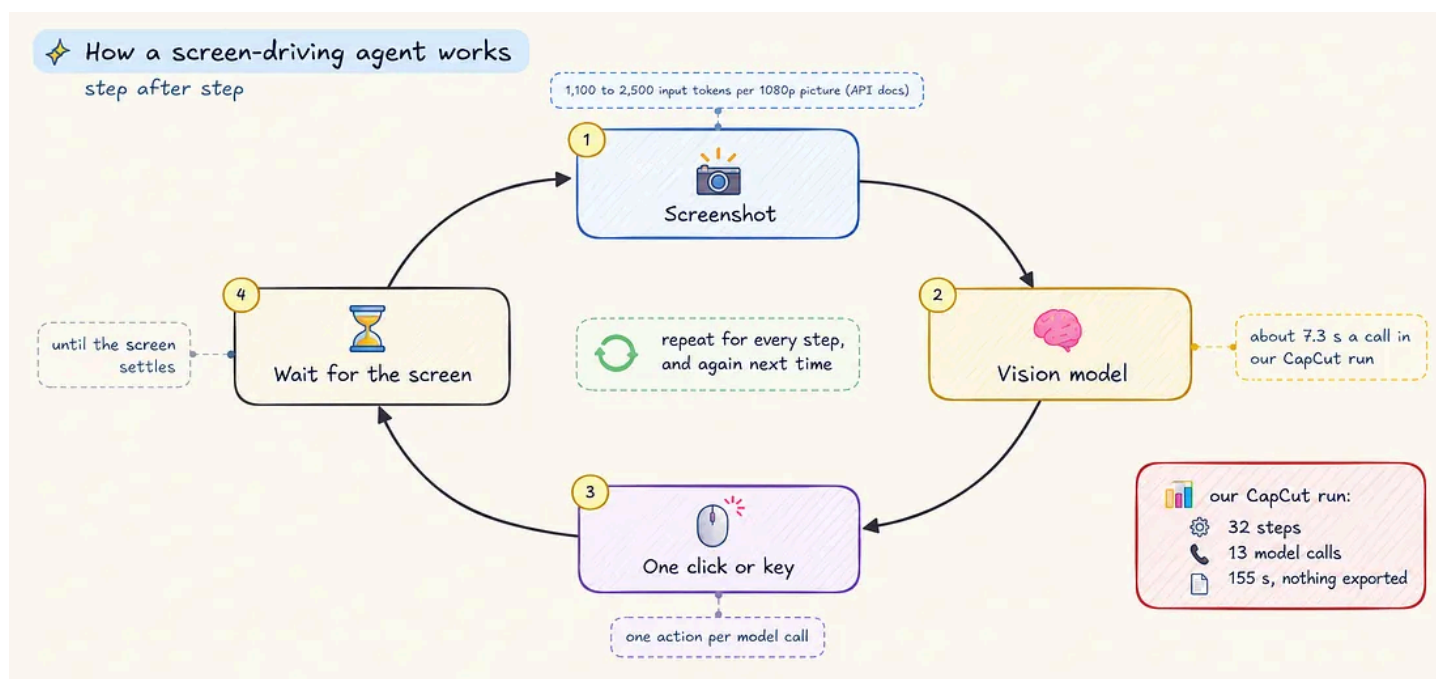
```
|— toolkit/ #уровень инструментов: реестр, результаты, цикл агента и цепочки
|— apps/ # инструменты для отдельных программ: Photoshop, Premiere, After Effe
|— office/ # инструменты для работы с электронными таблицами, слайдами и докум
|— cad/ # инструменты для черчения и поиска планов домов
```

- three/ # 3D-инструменты в Blender в фоновом режиме
- video/ # временная шкала и инструменты редактирования для CapCut и JianYing
- hub/ # инструменты обмена сообщениями для Slack, Gmail и других рабочих при
- desktop/ # оконные инструменты в дереве специальных возможностей и освоены

Профилирование экранного агента

Прежде чем создавать что-то новое, я хотел понять, на что уходит время экранного агента. 1 октября я дал своему первому экранному агенту небольшое задание: открыть CapCut, добавить видео в новый проект, применить к нему любой бесплатный фильтр и экспортировать. Он выполнил 32 шага и 13 вызовов модели и сдался через 155 секунд, ничего не экспортировав.

Я поступил с ней так же, как поступил бы с любой медленной системой: сначала измерил, нашел причину, выдвинул гипотезу, устранил проблему и снова измерил. Каждый из 32 шагов агента повторял один и тот же небольшой цикл.



Цикл экрана: скриншот, модель машинного зрения, один клик, ожидание появления экрана и так далее (создано Фаридом Ханом)

Поскольку агент регистрирует каждый шаг, мы можем разделить 155 секунд на отрезки.

ВЫВОД

13 вызовов модели 95,4 с 61,5%

действие и ожидание **для** CapCut 37,1 с 23,9%

ожидание **для** стабилизации экрана 7,7 с 5,0%

чтение окна 4,8 с 3,1%

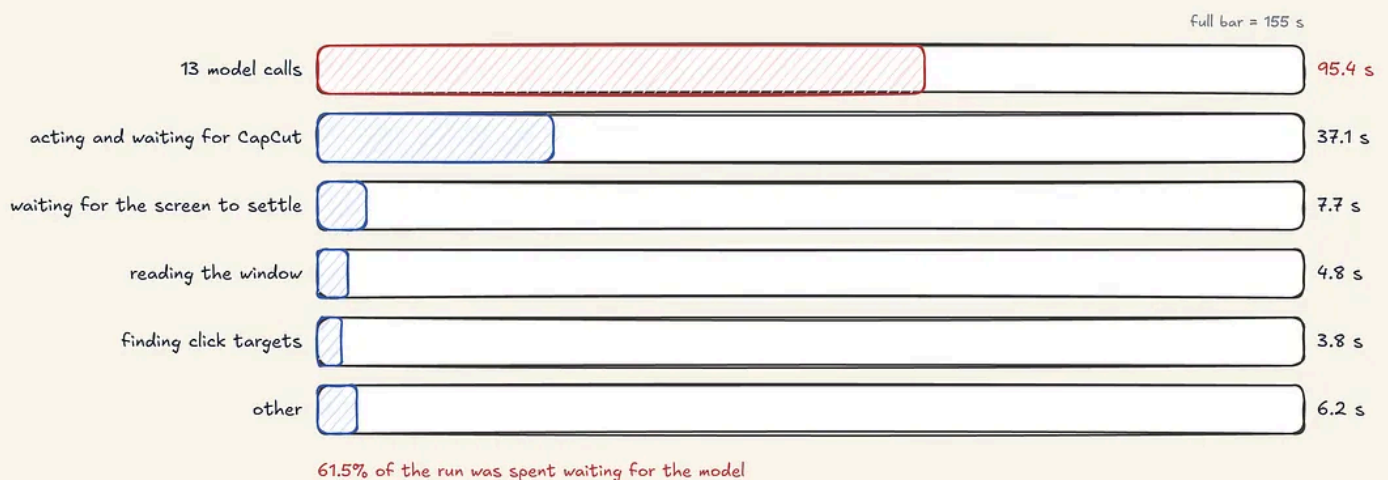
поиск целей для клика 3,8 с 2,5%

прочее 6,2 с 4,0%

входные токены: от 29 900 до 41 600

Вызовы модели заняли **61,5% времени выполнения**, а количество входных токенов, использованных в заданиях, составило от 30 000 до 42 000.

where 155 seconds went in a failed CapCut run



155 секунд ушло на неудачный запуск CapCut, при этом на вызовы модели ушло 95,4 секунды (создано **Фаридом Ханом**)

$$T_{\text{task}} = \sum_{i=1}^N (t_{\text{model},i} + t_{\text{act},i} + t_{\text{settle},i})$$

A screen agent pays for a model call at every step, so its time grows with the number of steps N

Время выполнения задачи — это сумма времени на моделирование, действие и принятие решения на каждом этапе (создано **Фаридом Ханом**)

Эти звонки обрабатывались медленно, в среднем за 7,3 секунды, в то время как модель, которую я использую сейчас, обрабатывает такой же звонок за 1,6–2,7

секунды. Однако более быстрая модель сокращает только один член этой суммы, а количество шагов остается прежним.

Чтобы обеспечить объективное сравнение запусков, мы ведем журнал, в котором для каждой задачи есть отдельная строка, начиная с неудачного запуска.

```
# speed_ledger.py: вызовы модели, вызовы инструментов, действия в видимом окне,
ledger = Ledger()
ledger.add("Фильтр CapCut, агент экрана", 13, 0, 32, "30k-42k", "155 с", "не уда
ledger.show()
```

```
#### ВЫВОД ####
инструменты для работы с моделями задач, токены пользовательского интерфейса, п
Фильтр CapCut, агент экрана 13 0 32 30 000-42 000 155 с сбой
```

Шесть издержек экранного цикла

Один запуск мало что доказывает, поэтому я сравнил наши результаты с тем, что исследователи получили при работе с другими агентами, использующими скриншоты. Во всех этих исследованиях фигурируют шесть издержек.

1. **Каждое действие требует нового плана.** В OSWorld-Human планирование и рефлексия занимали от 76% до 96% времени одного сильного агента, а лучшие агенты делали в 2,7–4,3 раза больше шагов, чем требовалось.
2. **Каждый звонок — это картина.** Снимок экрана в разрешении 1080p стоит от 1100 до 2500 входных токенов, а в ScreenSpot-Pro область, по которой нужно кликнуть, занимает всего 0,07 % снимка.
3. **Каждое действие ожидает завершения загрузки программы.** Изображение не может показать, когда программа завершила загрузку, поэтому агент приостанавливает работу и проверяет снова. В нашем тесте это заняло 44,8 секунды.
4. **Агент использует ваш экран.** Пока вы ждете, программа выполняет одну задачу за раз, и когда авторы OSWorld уменьшили размер окна, процент успешных запусков упал с 50,79 % до 15,04 %.

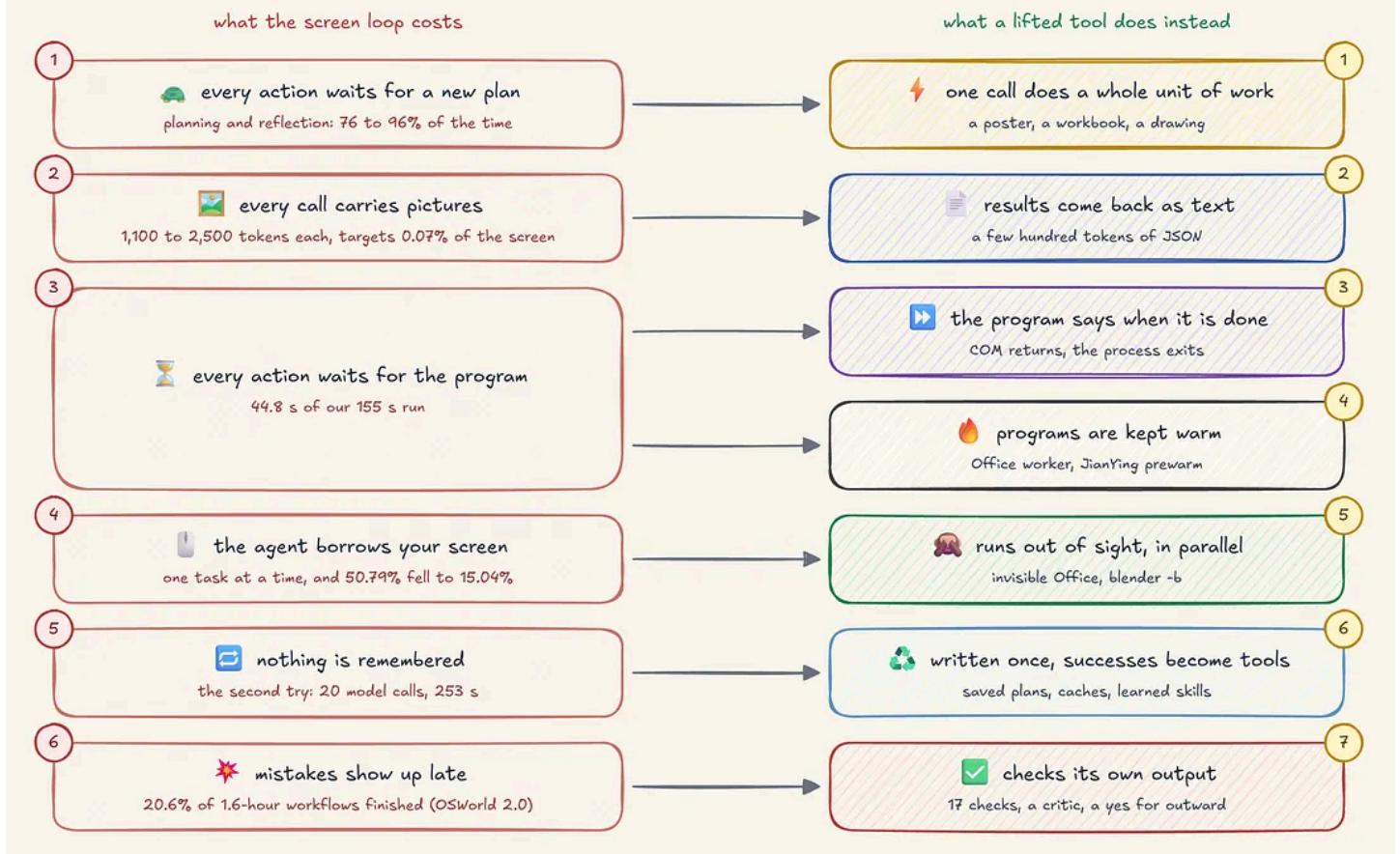
5. **Ничего не запоминается.** При каждом запуске программа исследует программу заново. Наша вторая попытка с CarCut оказалась хуже первой: 20 вызовов модели и 253 секунды.
6. **Ошибки проявляются слишком поздно.** Ошибки накапливаются при выполнении длительных задач, потому что скриншот не может подтвердить правильность результата. В 1,6-часовых рабочих процессах OSWorld 2.0 лучший агент справился на 20,6 %.



The screen agent paid 155 s and then 253 s, while a learned task repeats in 3.0 s (Created by [Fareed Khan](#))

Screen agents are improving fast: on OSWorld-Verified the best now finish 86 to 90% of short desktop tasks. Better models shrink only the first two costs, though. The other four come from the interface, which is the part our agent changes. Each cost points at a property our tools must have.

six costs of the screen loop, and the tool property that removes each one

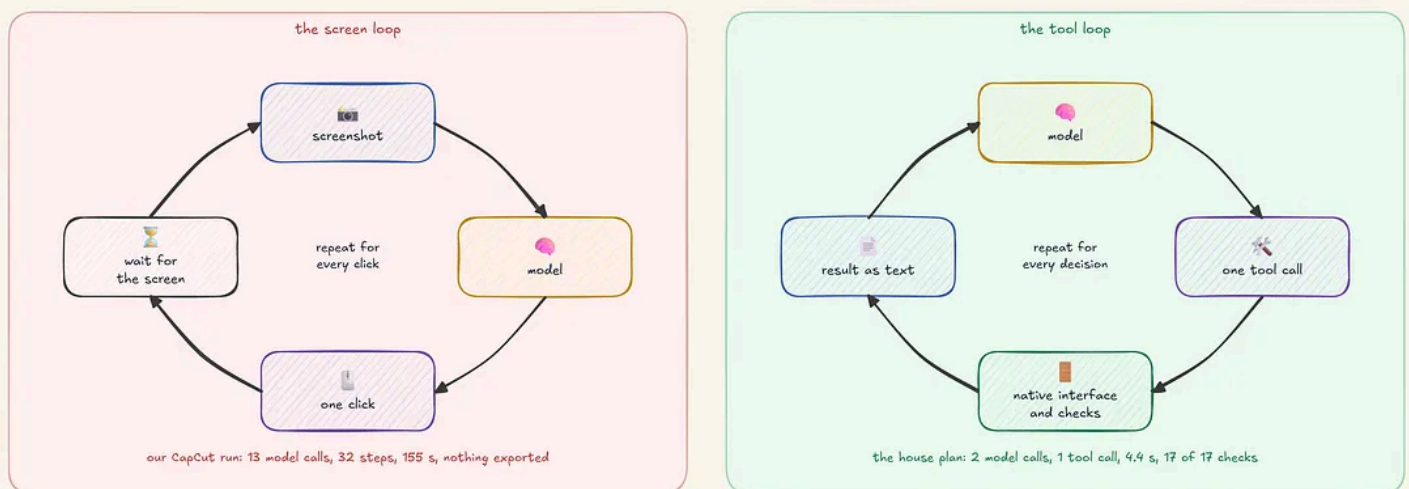


The six costs of the screen loop, and the property of a lifted tool that removes each one (Created by **Fareed Khan**)

Designing the Tool Layer

Most of those costs trace back to one design decision: the model meets the computer through a screen. Interface lifting replaces the screen with typed functions that the model calls directly.

a model call for every click, or a model call for every decision



The screen loop pays a model call for every click, and the tool loop pays one for every decision (Created by Fareed Khan)

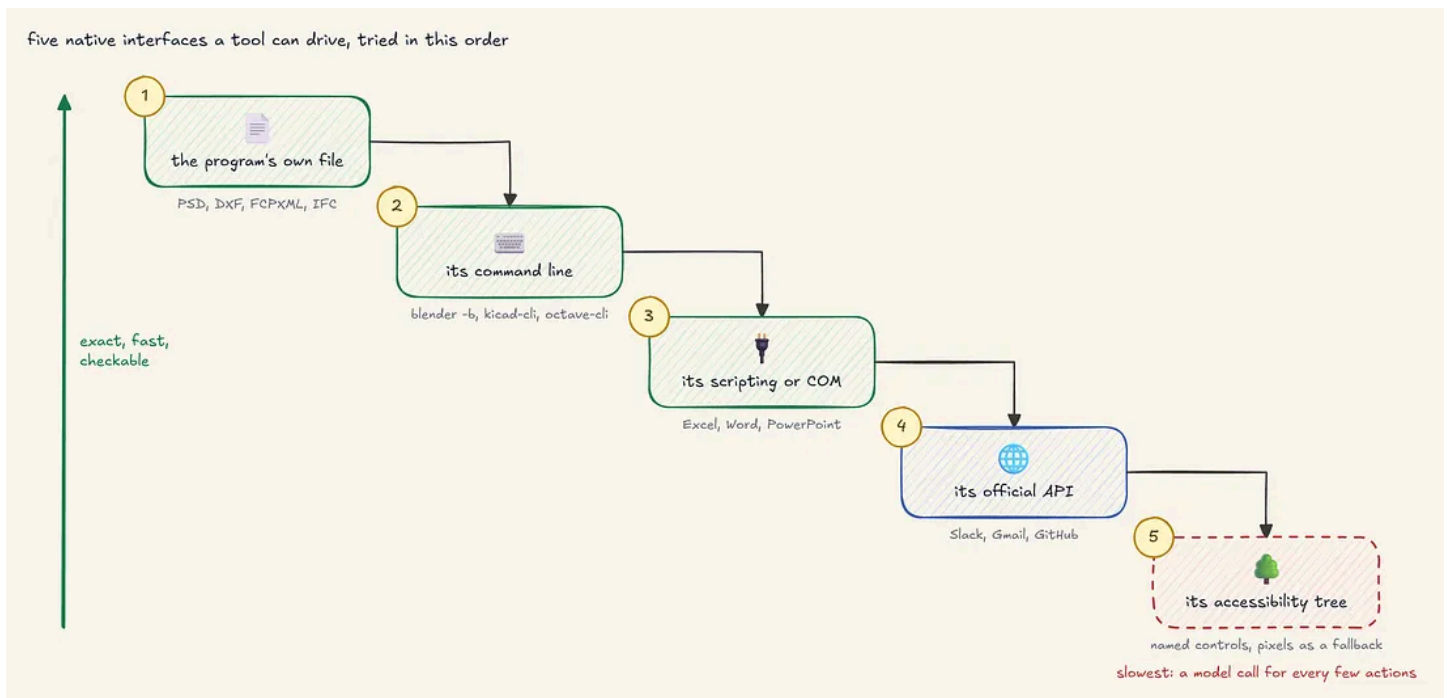
Four testable predictions

The hypothesis is that **the interface is the bottleneck, not the model**. If that is right, the ledger should show four things for the same model on the same kind of work.

1. **Model calls per task fall from tens to a handful**, because one tool call does a whole unit of work.
2. **Input tokens per task fall by an order of magnitude**, because results come back as a few hundred tokens of text.
3. **Errors surface at the step that made them**, because each tool checks its own output before it returns.
4. **The person keeps the screen**, because the tools run out of sight.

Five native interfaces

Most programs have a better interface than their screen. Our tools use five kinds, in this order of preference.

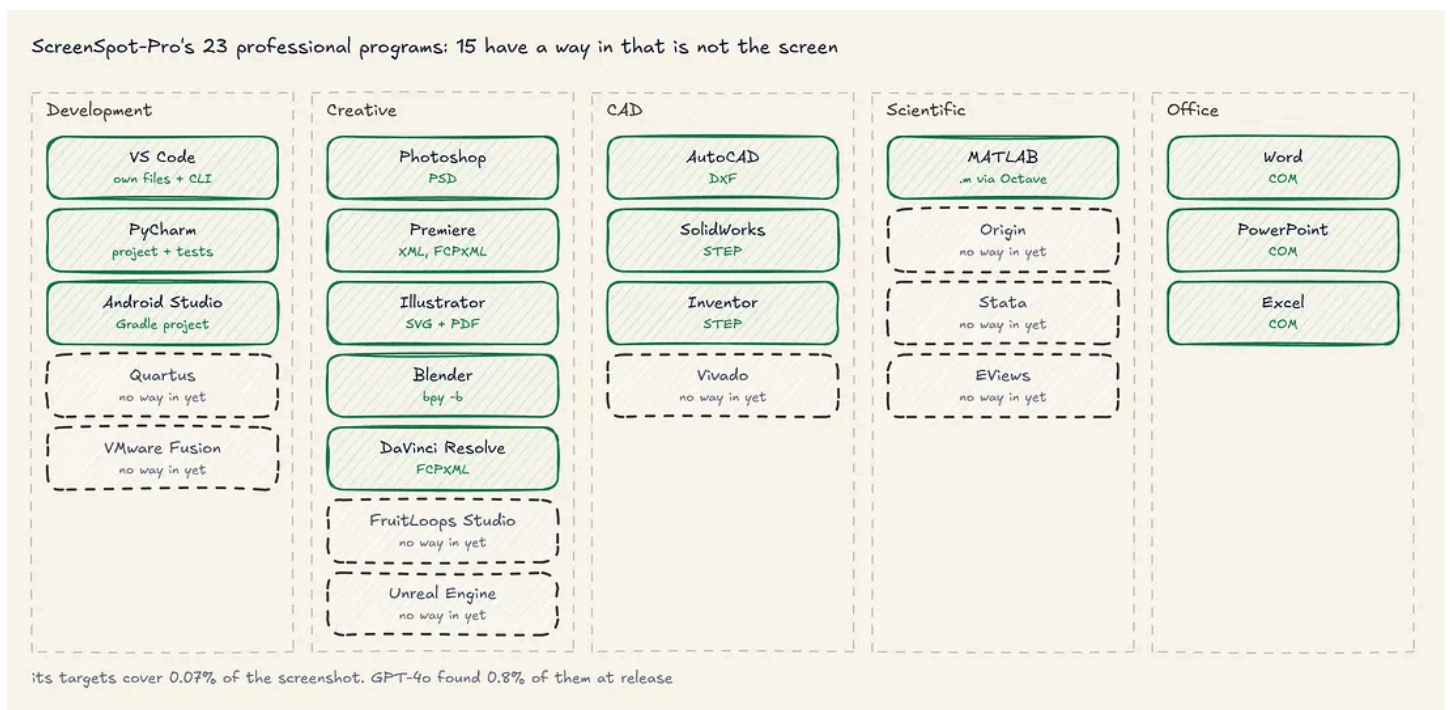


Five native interfaces a tool can drive, from the program's own file down to its accessibility tree (Created by Fareed Khan)

1. **The program's own file**: Photoshop's PSD, AutoCAD's DXF or an FCPXML timeline, opened as if a person had made it.

2. **Its command line:** `blender -b` renders without a window.
3. **Its scripting or COM interface:** Excel recalculates a workbook we wrote and hands back the computed values.
4. **Its official API:** Slack takes an upload and answers with the file's id.
5. **Its accessibility tree:** the named controls of any window, with pixels only as a fallback.

The first four give results a tool can check exactly, because the tool knows what it wrote. The price is work up front, since someone has to write each tool and its checks. For 15 of ScreenSpot-Pro's 23 programs, one of those first four interfaces exists.



ScreenSpot-Pro's 23 professional programs, with the 15 that have a native interface other than the screen (Created by [Fareed Khan](#))

When CoAct-1 let an agent write code next to its clicks, the average task dropped to 10.15 steps, “compared to 15 for leading GUI agents”. The top entry on the OSWorld-Verified leaderboard mixes code and clicks too.

The tool contract

Every tool in our agent keeps the same four-part contract, and the model only ever reads the first part.

1. **A schema:** a name, a docstring and typed arguments.

2. **An implementation:** code that drives the program through its native interface.

3. **A verifier:** checks on the output, sent back with the result.

[Открыть в приложении ↗](#)

[Регистрация](#)

[Вход](#)

Medium

 Search



A decorator reads the schema off the function itself. Each tool is written once, as ordinary Python.

```
# toolkit/registry.py: what the model sees of the computer, and what runs when
import inspect
import typing

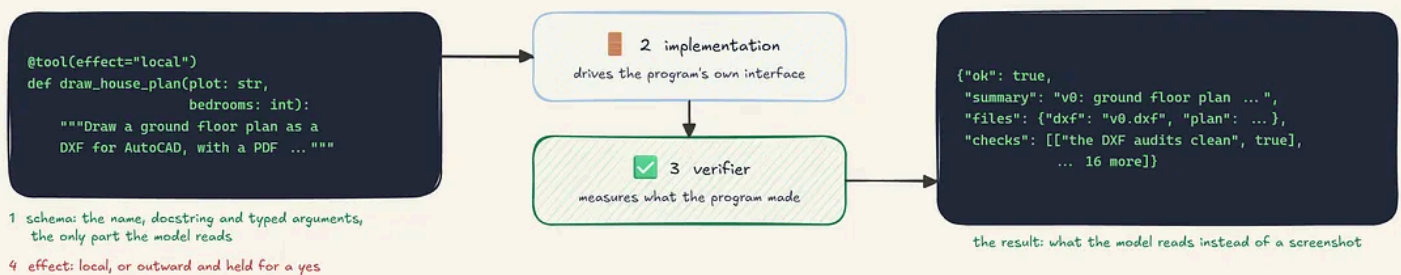
TOOLS = {}
JSON_TYPES = {str: "string", int: "integer", float: "number", bool: "boolean", ...}

def tool(effect="local"):
    """Register a function as a tool. Its name, docstring and type hints become
    def register(fn):
        hints, params = typing.get_type_hints(fn), inspect.signature(fn).parameters
        props = {n: {"type": JSON_TYPES[typing.get_origin(hints[n]) or hints[n].__origin__]}
        TOOLS[fn.__name__] = {
            "fn": fn,
            "effect": effect, # "outward" tool
            "schema": {"name": fn.__name__,
                      "description": inspect.getdoc(fn), # the docstring
                      "parameters": {"type": "object", "properties": props,
                                     "required": [n for n, p in params.items()
                                     ]}
        }
        return fn
    return register
```

Each tool also returns the same shape, which is all the model sees of the result.

```
# toolkit/result.py: the only thing a tool sends back to the model
def result(summary, files=None, checks=(), **extra):
    checks = [[name, bool(passed)] for name, passed in checks]
    return {"ok": all(passed for _, passed in checks), "summary": summary,
           "files": {k: str(v) for k, v in (files or {}).items()}, "checks": checks}
```

the contract every tool keeps



The tool contract: a schema the model reads, an implementation on a native interface, a verifier, and an effect
(Created by [Fareed Khan](#))

Project setup

We will build the agent as one Python package for Windows 11, and everything it installs stays inside the project folder.

```
uv init --python 3.12 agent && cd agent      # one project folder for everything
uv add openai pywin32 comtypes psutil pillow numpy ezdxif ifcopenshell
```

`openai` is only the client. Any OpenAI-compatible provider or a local model works, and every model call in this post runs on GLM-5.3-Flash.

The agent runtime

The model reads the request and the tool schemas, then either answers or asks for tool calls. The runner executes each call, holds anything outward for a yes, and sends the results back as text.

```
# toolkit/loop.py: the model decides, tools act, results come back as text
import json

def run_agent(request, model, tools, row, max_turns=8):
    messages = [{"role": "system", "content": SYSTEM}, {"role": "user", "content": request}]
    schemas = [{"type": "function", "function": t["schema"]} for t in tools.values()]
    for _ in range(max_turns):
        reply = model.chat(messages, tools=schemas)  # one model call
        row.model_calls += 1
        if not reply.tool_calls:
            return reply.content  # the model has answered
        messages.append(reply.message)
        for call in reply.tool_calls:
            t = tools[call.name]
            if t["effect"] == "outward" and not approved(call):
                out = {"ok": True, "pending": preview(call)}  # anything other
```

```

else:
    out = t["fn"](**call.args) # the program c
    row.tool_calls += 1
    messages.append({"role": "tool", "tool_call_id": call.id, "content":
return "stopped after too many turns"

```

The runner prints a line for each model call and each tool call, which turns each output below into a trace. Before any tool exists, we send it the request a shop owner might send a designer before Eid.

```

request = "photoshop file 1080x1080: background shop.jpg; logo logo.png top-left
print(run_agent(request, model, TOOLS, row)) # TOOLS is still empty

```

```

#### OUTPUT ####
model 1.1 s in 412 tok -> reply
I cannot make a Photoshop file yet: none of my tools writes PSD files. I can list
total 1.1 s, 1 model call, 0 tool calls

```

The model understood the request but had no tool to act with yet. The first tool we build is the one this request needs.

Layer Compositor

A designer who gets that Eid Sale request sends back a PSD, and the point of a PSD is that every part of the poster is still a named layer the client can move.

By screenshots, that is about 40 actions: the new-document dialog, placing the photo and the logo, two text layers, two colours, four positions, four layer names and the save dialog. Photoshop sits in ScreenSpot-Pro's creative category, where targets are tiny. Each of those actions is a model call carrying a screenshot, and all 40 have to go right.

tokens $\geq 40 \times (2,300 \text{ to } 3,200) = 92,000 \text{ to } 128,000$

A 4-layer poster by clicks is about 40 actions, and each one is a model call carrying a screenshot

At least 40 model calls times 2,300 to 3,200 tokens is 92,000 to 128,000 input tokens for one poster (Created by [Fareed Khan](#))

Because the PSD format is documented, we can expose layered composition itself as a tool. The model fills in the arguments, and the docstring is all it ever learns about the tool.

```
# apps/photoshop.py
@tool()
def compose_layered_image(size: list, layers: list, name: str = "poster") -> dict:
    """Make a layered PSD that opens in Photoshop, GIMP, Photopea and Affinity Designer.
    size: [width, height] in pixels. layers: bottom first, each {"name", "kind": "text", "at": "fill", "top-left", "center" ..., "color": a name or #rrggbb}
    and checks read back from the saved file."""
    placed = [place_layer(spec, size) for spec in layers] # pictures fitted
    path = OUT / "psd" / f"{slug(name)}.psd"
    flat = write_psd(path, size, placed) # the file, written
    return result(f"PSD with {len(placed)} layers ({', '.join(n for n, _ in placed)}, color: {', '.join(c for _, c in placed)}, files={\"psd\": path, \"preview\": save_preview(flat, path)}), checked")
```

`place_layer` turns each description into a picture and a position, and `write_psd` writes the file: a header, a record for each layer, the raw pixels of each layer, and the flattened picture Photoshop shows before it reads any layer.

```
# apps/photoshop.py (continued)
def write_psd(path, size, layers):
    """layers: [(name, RGBA image, (left, top))], bottom first. Writes an 8-bit
    w, h = size
    canvas = Image.new("RGBA", size, (255, 255, 255, 255)) # stacked images
    records, channel_data = b"", b""
    for name, im, (left, top) in layers:
        im = im.convert("RGBA")
        canvas.alpha_composite(im, (max(0, left), max(0, top)), (max(0, -left),
```

```

lw, lh = im.size
r, g, b, a = im.split()
chans = [(-1, a), (0, r), (1, g), (2, b)] # channels
rec = struct.pack(">iiiiH", top, left, top + lh, left + lw, 4) # the layer box
for cid, _ in chans:
    rec += struct.pack(">hI", cid, 2 + lw * lh) # each channel's box
extra = struct.pack(">I", 0) + struct.pack(">I", 0) + _pascal(name) + _pascal(' ')
records += rec + b"8BIMnorm" + bytes([255, 0, 0, 0]) + struct.pack(">I", 0)
channel_data += b"".join(struct.pack(">H", 0) + ch.tobytes() for _, ch in chans)
info = struct.pack(">h", len(layers)) + records + channel_data
info += b"\0" * (len(info) % 2) # this is for alignment
lmi = struct.pack(">I", len(info)) + info + struct.pack(">I", 0) # layer metadata
flat = canvas.convert("RGB")
data = b"8BPS" + struct.pack(">H6sHIIHH", 1, b"\0" * 6, 3, h, w, 8, 3) # 8-bit per sample
data += struct.pack(">II", 0, 0) + struct.pack(">I", len(lmi)) + lmi # no compression
data += struct.pack(">H", 0) + b"".join(c.tobytes() for c in flat.split())
Path(path).write_bytes(data)
return flat

```

The two helpers store each layer's name as a short Pascal string and again in a Unicode block tagged `luni`, which is where Photoshop reads names from.

The verifier, `check`, opens the saved file again with Pillow's PSD reader, which knows nothing about our writer. It compares each layer's name and box, then the flattened picture, pixel for pixel.

Importing the module registers the tool. Now we send the same request again.

```

from apps import photoshop # registers compose_layered_image

print(run_agent(request, model, TOOLS, row))

```

```

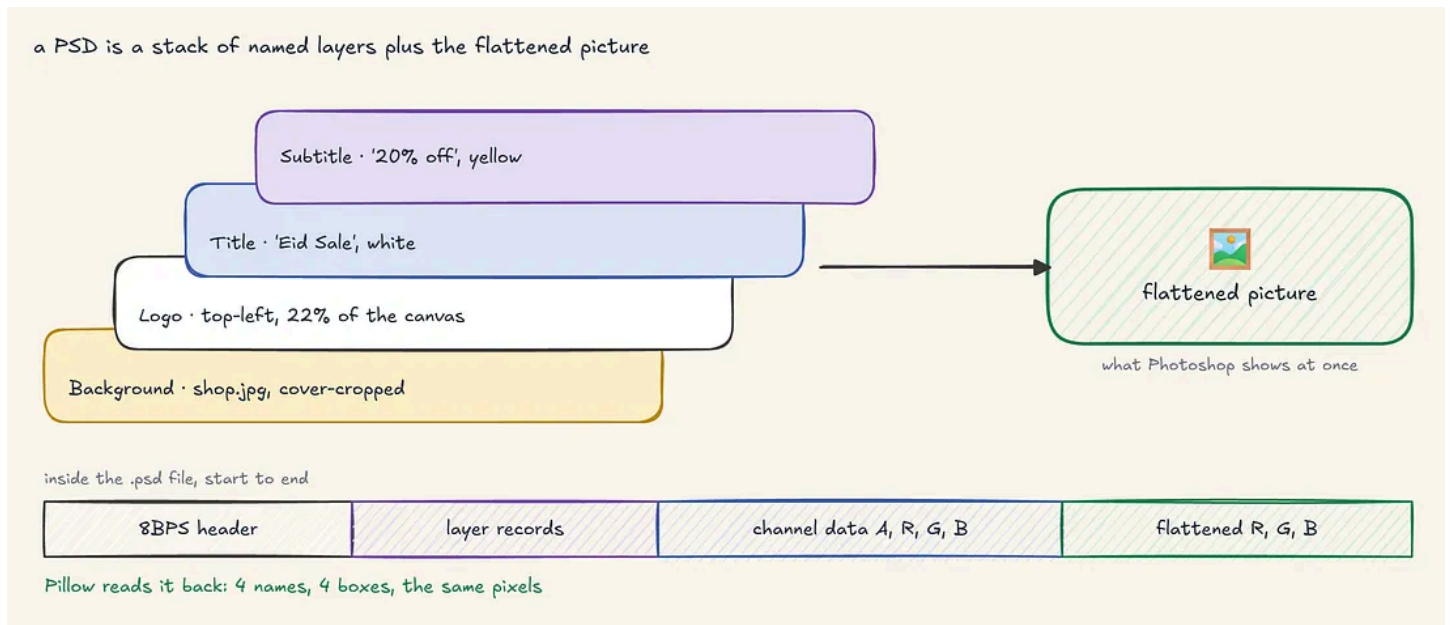
#### OUTPUT ####
model 1.4 s in 1,384 tok -> compose_layered_image(size=[1080, 1080], name="Eid Sale",
{"name": "Background", "kind": "image", "file": "background.png"},
{"name": "Logo", "kind": "image", "file": "logo.png"},
{"name": "Title", "kind": "text", "text": "Eid Sale"},
{"name": "Subtitle", "kind": "text", "text": "Eid Sale"}

tool 0.6 s <- ok, 2/2 checks, out\psd\Eid_Sale.psd
model 0.9 s in 1,731 tok -> reply
PSD with 4 layers (Background, Logo, Title, Subtitle), 1080x1080: out\psd\Eid_Sale.psd
Opens in Photoshop, GIMP, Photopea and Affinity with every layer movable. Check

```

name, in its place; the flattened picture is the layers stacked.
total 2.9 s, 2 model calls, 1 tool call, 0 screenshots, 3,115 input tokens

The model made one decision, the whole poster, and the tool did 40 actions' worth of work behind it. **Two model calls and about 3,100 input tokens** replace about 40 calls and 92,000 or more.



The Eid Sale poster's four layers, bottom to top, beside the flattened picture and the file's byte layout (Created by **Fareed Khan**)

```
ledger.add("layered image, Eid Sale poster", 2, 1, 0, "3,115", "2.9 s", "2/2")
```

Workbook Engine

The next request asks more of the model: “marks sheet in Excel for 8 students with totals, grades and a class summary”. Someone has to invent the students and decide the columns, and by screenshots the sheet is well over a hundred actions. Microsoft’s UFO2 paper says long GUI sequences, “e.g. iteratively selecting and formatting cells in Excel”, “can often be collapsed into a single API call”.

Our spreadsheet tool takes a whole workbook design as one argument. The model does all of its thinking while it writes that argument, and the docstring keeps it away from the part models get wrong: cell addresses.

```
# office/tools.py
@tool()
def build_workbook(plan: dict) -> dict:
    """Build an Excel workbook with live formulas, recalculated by Excel itself
    plan: {"title", "sheets": [{"name", "kind": "table" or "summary", ...}]}.
    NEVER write cell addresses (A1, C2:C9) or formulas with them. Computed column
    "[Qty] * [Price]", "ROUND([Obtained] / [Total] * 100, 1)". Totals, metrics and
    writes every formula, so they stay live when the client edits the data."""
    path, expected = write_book(plan) # formulas from the plan
    excel = run_com_job("excel", path, read=expected) # Excel recalculates,
    return result(f"Made {path.name} and its PDF {path.with_suffix('.pdf').name}")
    files={"xlsx": path, "pdf": path.with_suffix(".pdf")}, checks=
```

When I ran it on 5 October 2026, GLM-5.3-Flash put this table sheet into the `plan` argument, shown here trimmed. A second sheet declares four summary metrics and two charted groups.

```
{"name": "Marks", "kind": "table",
  "columns": ["Roll No", "Student Name", "Maths", "Science", "English", "History",
    {"name": "Total", "formula": "[Maths] + [Science] + [English] + [History]"},
    {"name": "Percentage", "type": "percent", "formula": "ROUND([Total] / 4, 1)"},
    {"name": "Grade", "formula": "IF([Percentage] >= 90, \"A+\", IF([Percentage] >= 80, \"A\", IF([Percentage] >= 70, \"B\", IF([Percentage] >= 60, \"C\", \"D\")))"}],
  "rows": [[1, "Aarav Sharma", 88, 92, 79, 85, null, null, null], "... 7 more students"],
  "total": {"Maths": "average", "Science": "average", "English": "average", "History": "average"}}
```

Writing that argument is the slowest model call in the task. Model latency also has long tails: on 1 October, the median answer time of DeepSeek-V4.1-Flash jumped to 39.8 seconds, against its usual 2 to 3.5. So every model call has a waiting time, `hedge_s`, after which an identical backup request goes out and the first answer wins. At \$0.15 and \$0.50 per million input and output tokens, a rare backup is cheap insurance.

Inside the tool, `write_book` turns column names in brackets into cell references, and a second function, `py_value`, computes the same expressions in Python with the `ast` module. Every number Python can compute becomes something Excel has to agree with.

Total	G2	=C2+D2+E2+F2
Percentage	H2	=ROUND(G2/400*100,1)
Grade	I2	=IF(H2>=90,"A+",IF(H2>=75,"A",IF(H2>=60,"B",IF(H2>=50,"C",IF(H2>=40,"D",IF(H2>=30,"F",IF(H2>=20,"G",IF(H2>=10,"H",IF(H2>=0,"I"))))))))

Then Excel does the maths. The COM job opens a new, invisible Excel that belongs to us, recalculates, scans every used cell for error codes, reads the cells we have expectations for and prints the PDF.

`DispatchEx` always starts a new instance, never the Excel you have open, and the job runs in a child process started with `CREATE_NO_WINDOW`. When `CalculateFull` returns, Excel has finished, and the verifier compares each value Excel computed with Python's.

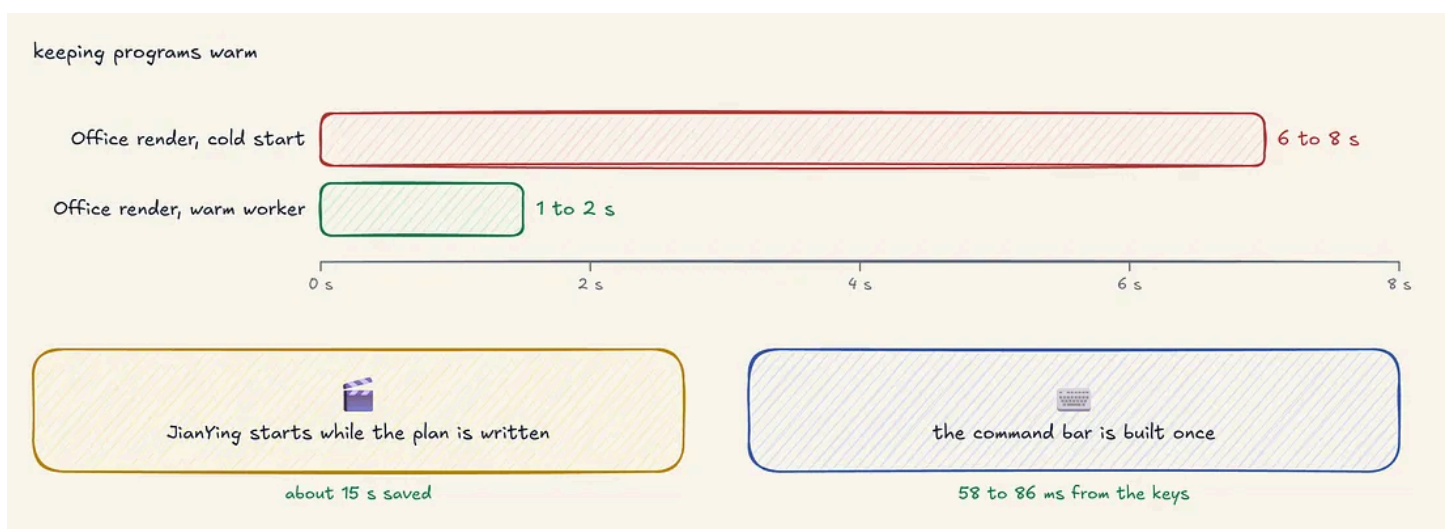
Each total Excel computes has to match the one Python computed from the same plan, within half a percent

```
model 8.6 s in 2,284 tok -> build_workbook(plan={"title": "Class Marks Sheet"
```

```
tool    9.2 s      <- ok, 2/2 checks, class-marks-sheet.xlsx, class-ma
pass    global    Excel recalculated 2 sheet(s): no e
pass    values    all 19 totals and summaries Excel c
model   1.1 s  in 4,631 tok -> reply
Made class-marks-sheet.xlsx and its PDF class-marks-sheet.pdf. Ask for changes,
total   18.9 s, 2 model calls, 1 tool call, 0 screenshots, 6,915 input tokens
```

Most of that time is real work: 8.6 seconds of the model writing the plan and 9.2 seconds of the tool driving Excel.

I also put a small command bar in front of the agent, opened with Ctrl+Alt+Space, and the screenshots in this post come from it.



time with the plan replayed from a recorded answer and Office's visual check off (Created by [Fareed Khan](#))



Microsoft Excel

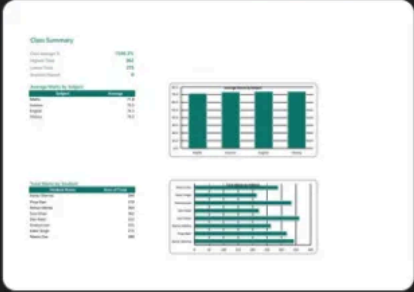
"marks sheet in Excel for 8 students with totals, grades and a class summary"

✦ | Ask your PC to do anything... (hold Ctrl+Alt+Space to talk) 🔗 🎤 ➤

You 6:20 PM
marks sheet in Excel for 8 students with totals, grades and a class summary

WORD, POWERPOINT, EXCEL AND PDF

Made class-marks-sheet.xlsx and its PDF class-marks-sheet.pdf. Ask for changes, or 'undo'.



class-marks-sheet.xlsx 📁 📄 class-marks-sheet.pdf 📁 📄 office_edited.xlsx 📁

9.2 s

Done in 9.2 s Enter send · Esc hide · Ctrl+N new chat ⭐ +

The marks sheet in the command bar, 9.2 seconds of toWarm programs: an Office render drops from 6 to 8 seconds to 1 to 2, and JianYing starts while the plan is written (Created by [Fareed Khan](#))

```
ledger.add("spreadsheet, marks sheet", 2, 1, 0, "6,915", "18.9 s", "19/19")
ledger.add("slides, 10-slide deck", 3, 1, 0, "8,240", "22 s", "11/11")
```

A deck goes through a `build_deck` tool that takes the whole outline. PowerPoint then reports how much room each box's text takes. Overflow shows up as "slide 6: text runs out of its box (Body: 212 pt of text in 180 pt)" and gets fixed before anyone sees it. A 10-slide deck took 22 seconds with 3 model calls, passed 11 of 11 slide checks and cost \$0.0018. A 5-page Word report took about 45 seconds, with 6 sections written at a time.

Warm worker pool

A one-off render takes about 6 to 8 seconds, mostly Office starting and quitting. So the tools share one worker that keeps Word, PowerPoint and Excel open and takes one JSON job per line. With it, a render drops to about 1 to 2 seconds. If a job hangs, the worker kills only the Office processes it started, never a Word window you opened yourself.

Floor Plan Solver

House designers get this request all the time: “a 10 marla house with 3 bedrooms”. A marla is a unit of land area, usually 225 square feet, so this is a 35 by 65 foot plot. Drawing it in AutoCAD takes hundreds of commands: walls, door swings, windows, dimensions, room names and a title block, all at a scale.

The drafting tool never opens AutoCAD. It writes a DXF, the drawing format AutoCAD reads, in inches so AutoCAD gets the scale right, and headless Chrome prints it to PDF at true scale. The model only has to read the brief and fill in two arguments, because the tool accepts plot sizes the way people say them.

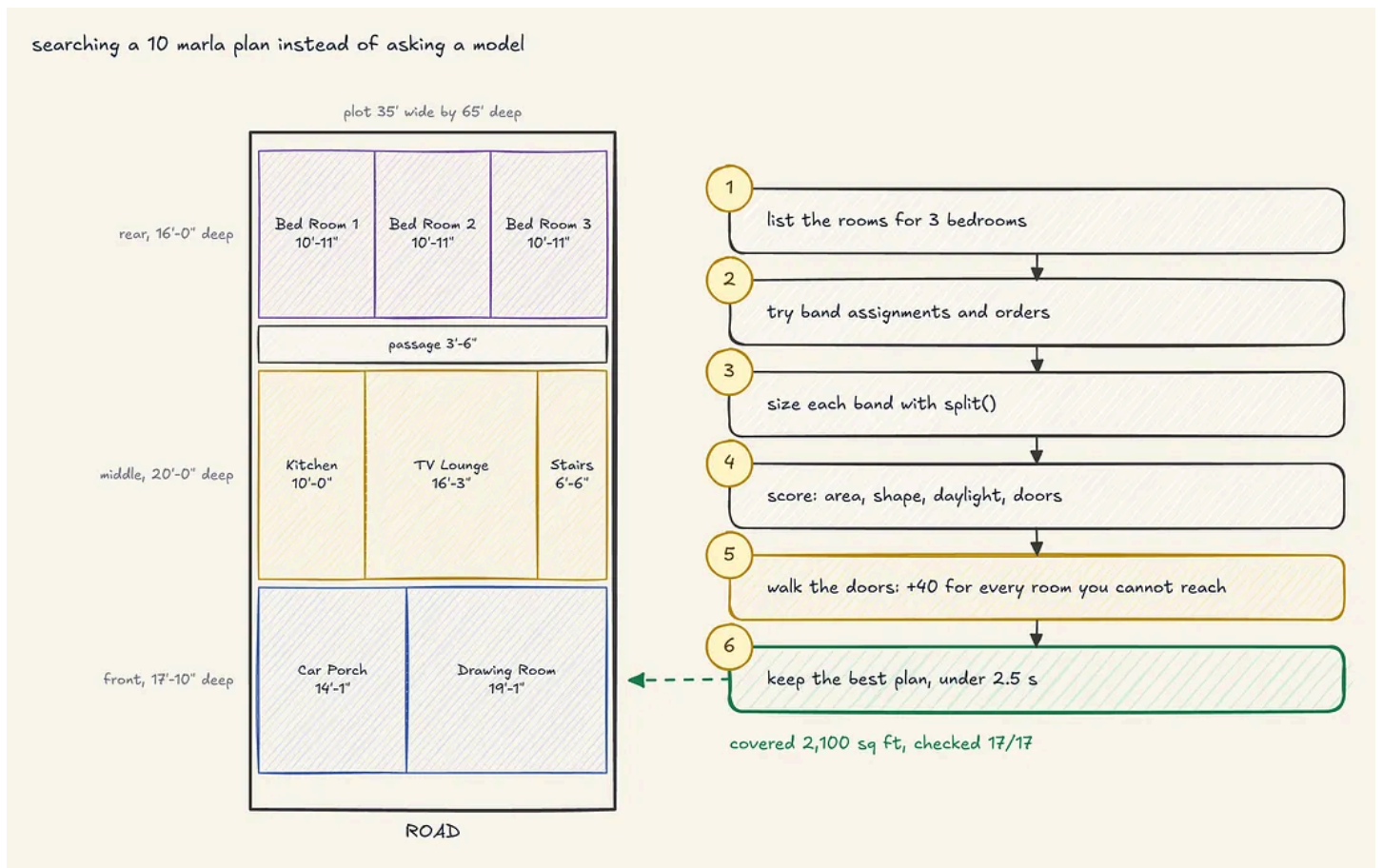
```
# cad/tools.py
PLOTS = {3: (20, 34), 5: (25, 45), 7: (30, 52), 10: (35, 65), 20: (50, 90)} #

@tool()
def draw_house_plan(plot: str, bedrooms: int, storeys: int = 1) -> dict:
    """Draw a ground floor plan as a DXF for AutoCAD, with a PDF at true scale.
    '35 x 65 ft'. A layout search places the rooms, doors and sheet scale, and
    returned. The saved plan file can be passed to the 3D and building-model tool
    w, d = plot_size(plot) # '10 marla' ->
    lay = plan({"plot": [w, d], "bedrooms": bedrooms, "storeys": storeys}) #
    return draw(lay) # DXF, PDF, the
```

The search inside `draw_house_plan` places rooms in bands from the road back: car porch and drawing room in front, kitchen, TV lounge and stairs in the middle, a passage, then bedrooms with attached baths.

Each candidate gets penalty points for rooms far from their target areas, corridor-shaped rooms, rooms that need daylight but have no outside wall, and missing doors between rooms that should share one. A room nobody can walk to from the entrance costs 40 points.

The tool keeps the best candidate it finds within 2.5 seconds, then picks the sheet scale: 1:125, because at 1:100 the plan needs 258 mm and A3 has 231.



Searching a 10 marla plan: rooms in bands from the road, each candidate sized, scored and walked through its doors
(Created by [Fareed Khan](#))

The agent ran the whole task in three steps.

```
#### OUTPUT ####
model  1.3 s  in 1,402 tok  -> draw_house_plan(plot="10 marla", bedrooms=3)
tool   2.1 s                                     <- ok, 17/17 checks, v0.dxf, v0.pdf, v0.png, v0.plan
model  1.0 s  in 1,866 tok  -> reply
v0: ground floor plan for a 35' x 65' plot (10.1 marla), 1:125 on A3. Covered 2
19'-1" x 17'-10"; ... Bed Room 3 10'-11" x 16'-0"; 3 attached bath(s). Notes: K
a skylight or a ventilator and an exhaust fan; ... Files: v0.dxf (AutoCAD), v0.p
total  4.4 s, 2 model calls, 1 tool call, 0 screenshots, 3,268 input tokens
```



AutoCAD drawing

"a 10 marla house with 3 bedrooms"

✦ | Ask your PC to do anything... (hold Ctrl+Alt+Space to talk) 🔗 🎤 ➤

You 6:39 PM
a 10 marla house with 3 bedrooms

HOUSE PLANS AND PARTS (CAD)

v0: ground floor plan for a 35' x 65' plot (10.1 marla), 1:125 on A3. Covered 2,100 sq ft. Rooms: Drawing Room 19'-1" x 17'-10"; Car Porch 14'-1" x 17'-10"; Kitchen 10'-0" x 20'-0"; Tv Lounge 16'-3" x 20'-0"; Stairs 6'-6" x 20'-0"; Passage 33'-6" x 3'-6"; Bed Room 1 10'-11" x 16'-0"; Bed Room 2 10'-11" x 16'-0"; Bed Room 3 10'-11" x 16'-0"; 3 attached bath(s). Notes: KITCHEN has no outside wall: give it a skylight or a ventilator and an exhaust fan; TV LOUNGE has no outside wall: give it a skylight or a ventilator. Files: v0.dxf (AutoCAD), v0.pdf, v0.png. (checked 17/17)



cad_edited.svg 📁 📄 cad_edited.pdf 📁 📄 cad_edited.png 📁 📄 v0.png

2.1 s

Done in 2.1 s Enter send - Esc hide - Ctrl+N new chat ⚙️ +

The 10 marla house in the command bar: DXF, PDF and preview in 2.1 seconds of tool time, checked 17 of 17
(Created by [Fareed Khan](#))

Because the search could not give the kitchen or the lounge an outside wall, the result says what a builder should add there. Its "(checked 17/17)" covers six rules about the plan, nine read-backs of the saved DXF and two checks on the printed sheet.

Refusals as results

A request like "make the kitchen bigger" becomes a call to a second tool, `resize_room`. It tries the kitchen a foot bigger and then six inches bigger, reruns the whole search each time with a 1.5-second budget, and keeps a version only if every plan rule still passes. When no version passes, it refuses and returns the reason.

```
#### OUTPUT ####
> make the kitchen bigger (a 5 marla plot)
model -> resize_room(room="kitchen", by_ft=1)
```

```

tool <- refused: in width, Bed Room 1 would lose its door; in depth, the rooms
The kitchen can't get bigger on this plot: in width, Bed Room 1 would lose its
it are already at their smallest. The plan stays as it is.

> scale 1:100                                (the 10 marla plan)
model -> set_sheet(scale="1:100")
tool <- refused: at 1:100 the plan needs 167 x 258 mm and A3 has room for 390
Couldn't draw that: at 1:100 the plan needs 167 x 258 mm and A3 has room for 390
for a bigger sheet.

```

The model can pass a refusal on as it is, or try the option it names, such as 1:125 on A3. We added the door rule after an early version took Bed Room 1's door away when it was asked for a kitchen 2 inches bigger.

```

ledger.add("drafting, 10 marla house plan", 2, 1, 0, "3,268", "4.4 s", "17/17")

```

Headless 3D and BIM Export

The search can produce a slightly different plan on another run. That is why the drafting tool also saves the exact layout as `v0.plan.json` and returns its path. The 3D and building-model tools take that path as their argument, which lets the model pass the file from one program to the next.

The 3D tool starts Blender from the command line in background mode, which has no window.

```

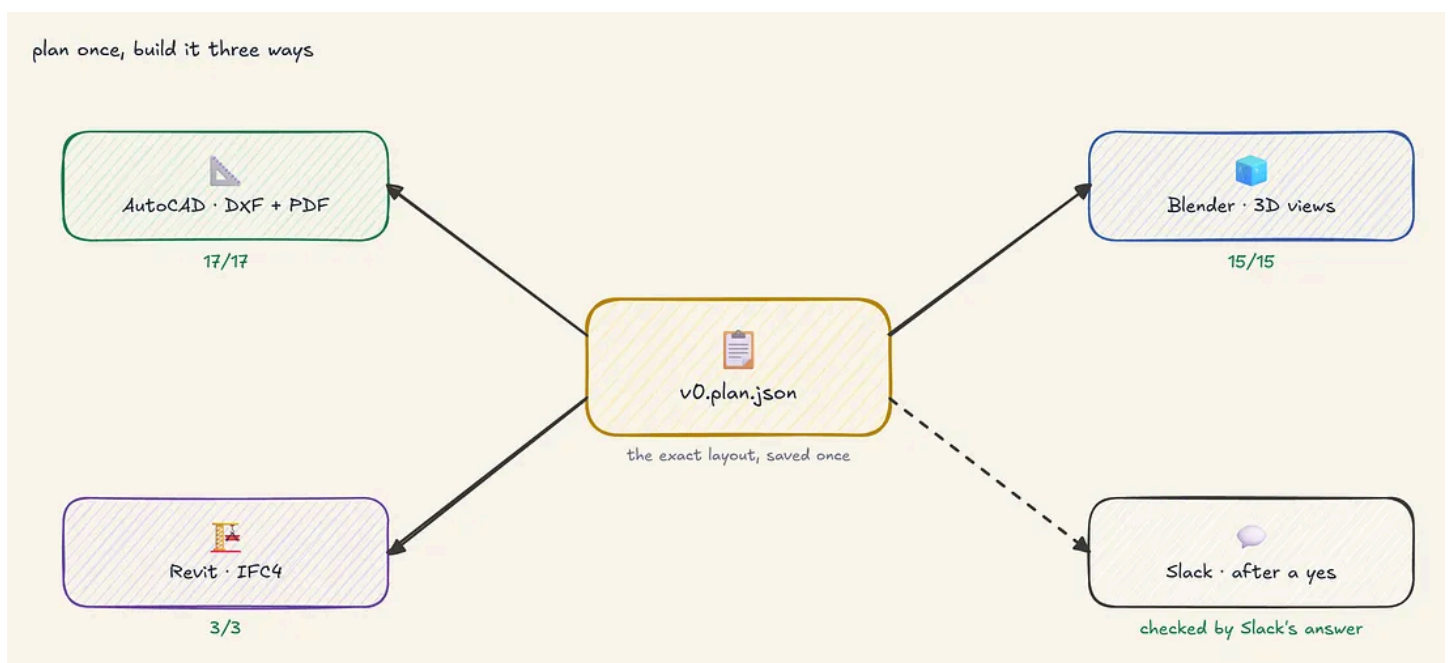
# three/blender.py
@tool()
def render_plan_3d(plan_file: str) -> dict:
    """Build a saved house plan as a 3D model in Blender and render a 3D floor plan
    spec = save_spec(house_spec(load_plan(plan_file))) # boxes
    cmd = [str(exe()), "-b", "--factory-startup", "--python-exit-code", "1",
           "-P", str(BL / "house_scene.py"), "--", str(spec), str(OUT_JSON)]
    p = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
    for raw in iter(p.stdout.readline, b''):
        if m := re.match(rb"PROGRESS (\d+)", raw): # the sci
            progress(int(m.group(1)))
    return result_from(OUT_JSON) # the

```

house_spec turns the plan into boxes in inches, with walls cut around each door and window. Blender renders both views with Eevee, plus a silhouette mask per view that lets the checks see whether the house got cut off. When the process exits, the render is done, and the tool reads the pictures and the checks from the result file.

```
#### OUTPUT ####
pass 3D: every room on the ground floor has its floor          9 of 9
pass 3D: glass in every ground-floor window
pass 3D: the ground-floor stairs reach the next floor          11.0 ft
pass 3D: the front door is in
... 11 more checks on the model and the two pictures
Checked: 15 of 15 passed.
Files: v0_plan3d.png, v0_front.png
```

Revit takes the same plan through IFC4, the open format for building models, with an export_building_model tool built on ifcopenshell. Its three checks reopen the file and count its elements, build every wall and slab as solid geometry on the plot, and run the IFC schema validator.



One saved plan feeds the AutoCAD drawing, the Blender views, the Revit model and the Slack post (Created by Fareed Khan)

None of these programs appears on your screen, and you keep working while the agent runs. Office runs as an invisible instance, Blender in background mode and JianYing parked past the right edge of the monitors. Programs that open a window

even from their command line, like MuseScore, run on a second Windows desktop that is never shown.



Four ways a program works out of sight while your screen, mouse and keyboard stay yours (Created by [Fareed Khan](#))

```
ledger.add("3D views of the plan", 2, 1, 0, "3,410", "33.8 s", "15/15")
ledger.add("building model, IFC4", 2, 1, 0, "3,190", "4.6 s", "3/3")
```

Cross-Program Orchestration

So far every request needed one program. This one needs three: “a 10 marla house with 3 bedrooms, then show the plan in 3d in blender and send it to slack #team”.

Two problems show up before any of them runs.

Schema shortlisting

The first problem is size. Our agent has over 200 tools at about 180 tokens of schema each. Sending all of them would add 36,000 tokens to every model call, and models choose worse from long lists, so a scorer decides which groups of tools go into the prompt.

```
# toolkit/shortlist.py
SIGNALS = {
    "hub": [(r"\bslack\b|\b(?:ms |microsoft )?teams\b|\btrello\b|\bnotion\b|\b|
```

```

"cad": [(r"\bhouse plan\b|\bfloor plan\b|\bmarla\b|\bkanal\b|\bbedrooms?\b|\b3d\b|\bthree[- ]d\b|\bblender\b", 10), (r"\brender(?:ing)?\b", 10)]
# ... 13 groups in all
}

def score(message, files=(), named=None):
    s = {}
    for group, sigs in SIGNALS.items():
        hits = sorted((w for pat, w in sigs if re.search(pat, message, re.I)), reverse=True)
        if hits:
            s[group] = hits[0] + 0.5 * (len(hits) - 1) # the strongest hit
    for k in {kind_of(f) for f in files}:
        for group, w in KIND_GROUPS.get(k, {}).items():
            s[group] = s.get(group, 0) + w # an attached picture
    if named: # the message name
        s["apps"] = max(s.get("apps", 0), 12)
    return s

```

When one group scores at least 5 and leads the next by 1.5, the choice is clear and only its tools go into the prompt. Otherwise the top three groups go in, and the model chooses among more tools.

$$\text{clear} \Leftrightarrow s_{(1)} \geq 5 \text{ and } s_{(1)} - s_{(2)} \geq 1.5$$

Only the best tool group goes into the prompt when it has 5 points and a lead of 1.5, otherwise the top three do

Only the best tool group goes into the prompt at a score of 5 or more and a lead of 1.5 or more (Created by Fareed Khan)

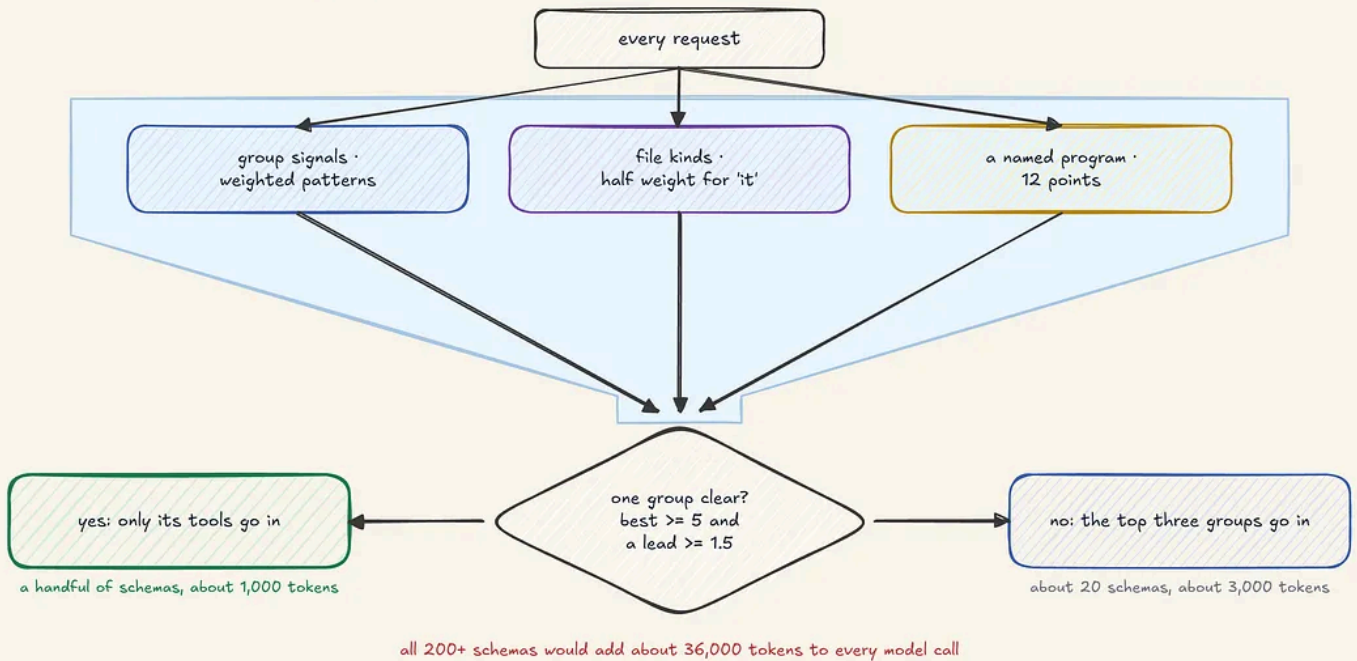
```

#### OUTPUT ####
'a 10 marla house with 3 bedrooms, then show the plan in 3d in blender and send
  hub 10.5   three 10.0   cad 7.0                                -> not clear: 3 groups
'photoshop file 1080x1080: ...' with shop.jpg and logo.png
  apps 12.0 (photoshop)   photo 7.0   sound 7.0                -> clear: 6 tools
'make it pop' in a new chat
  no group scores                                              -> 3 general tools

```

Naming a program is worth 12, enough that “background” and “subtitle” cannot pull the Photoshop request away. Our sentence does need three groups, and all three go in at about 3,200 tokens instead of 36,000.

which tools the model sees for one request



Which tools the model sees for one request: group signals, file kinds and named programs decide (Created by Fareed Khan)

Chain planning in one call

The second problem is round trips. Called one at a time, three dependent tools cost a model call before each tool and one more for the reply. The model already knows the whole chain when it reads the sentence, though. One more tool, `run_chain`, lets it write that chain in a single call, and a later call in the list can name an earlier call's output.

```
# toolkit/chain.py: one model call plans several dependent tool calls
@tool()
def run_chain(steps: list) -> dict:
    """Run tool calls in order. steps: [{"id", "tool", "args"}]. An argument wr
    that file from an earlier step's result. The chain stops at the first failed
    for a yes."""
    done = {}
    for s in steps:
        args = {k: resolve(v, done) for k, v in s["args"].items()} # "$pla
        t = TOOLS[s["tool"]]
        if t["effect"] == "outward" and not approved(s):
            return {"ok": True, "pending": preview(s["tool"], args), "done": do
        done[s["id"]] = out = t["fn"](**args)
        if not out["ok"]:
```

```

        return {"ok": False, "failed": s["id"], "done": done}      # stop when failed
    return {"ok": True, "done": done}

```

$$n_{\text{one at a time}} = k + 1 \quad n_{\text{chain}} = 2$$

k dependent tool calls take k + 1 model calls one at a time, and 2 when the model writes them as one chain

k dependent tool calls take k + 1 model calls one at a time, and 2 when the model writes them as one chain (Created by Fareed Khan)

The whole sentence takes one planning call.

```

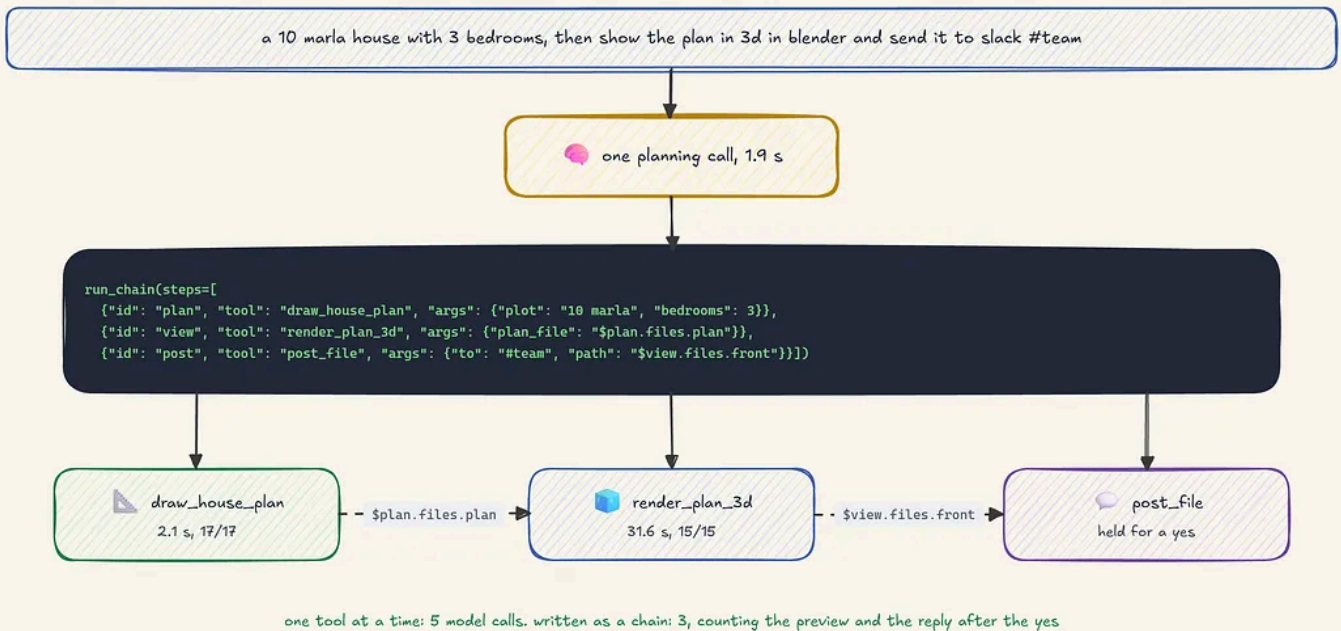
#### OUTPUT ####
> a 10 marla house with 3 bedrooms, then show the plan in 3d in blender and send
model  1.9 s in 3,672 tok -> run_chain(steps=[
    {"id": "plan", "tool": "draw_house_plan", "args": {}},
    {"id": "view", "tool": "render_plan_3d", "args": {}},
    {"id": "post", "tool": "post_file", "args": {"path": "v0_plan3d.png", "caption": "3D plan of a 10 marla house with 3 bedrooms."}}
])

tool  2.1 s          <- plan: ok, 17/17 checks
tool 31.6 s          <- view: ok, 15/15 checks, v0_plan3d.png, v0_front.png
tool  0.0 s          <- post: held for a yes
model  0.9 s in 4,355 tok -> reply
[Work apps] Ready to post v0_front.png to #team on Slack. Say 'yes' to go ahead

```

The model wrote the chain once, and the references carried the plan file and the render from one program to the next. Only the Slack post stopped, held at the gate.

one planning call, three tools, references instead of round trips



One planning call, three tools, with references in place of round trips (Created by [Fareed Khan](#))

The Approval Gate

Posting to Slack is the first step other people see. Its tool is registered as outward, and the runner never calls an outward tool without a yes.

```
# hub/tools.py
@tool(effect="outward")
def post_file(service: str, to: str, path: str) -> dict:
    """Post a file to a channel or a person on a work app (slack, teams). Others:
    out = SERVICES[service].upload(to, path) # each serv
    return result(f"Uploaded {Path(path).name} to {to}" + (" (checked)" if out["
        files={"link": out["link"]}, checks=[("the service names the
        undo=out["undo"]])
```

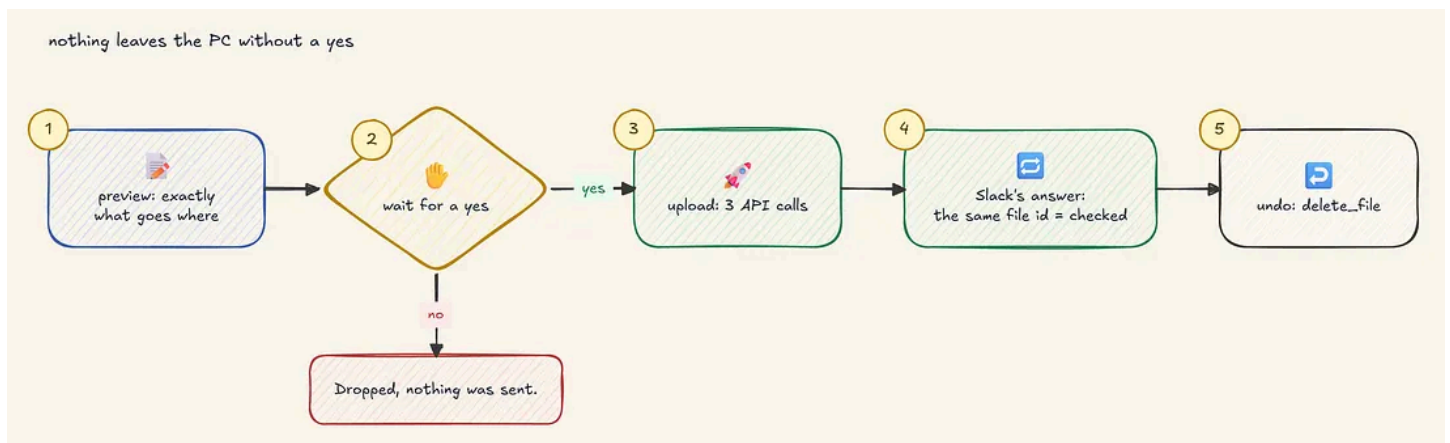
The preview is built from the held call itself and shows exactly what goes where. After a yes, the upload is three calls to Slack's Web API: an upload URL, the bytes and the share. The check passes only if Slack's answer names the file id it issued for our upload.

```
#### OUTPUT ####
> yes
tool 1.4 s <- post: ok, 1/1 checks
model 0.8 s in 4,618 tok -> reply
```

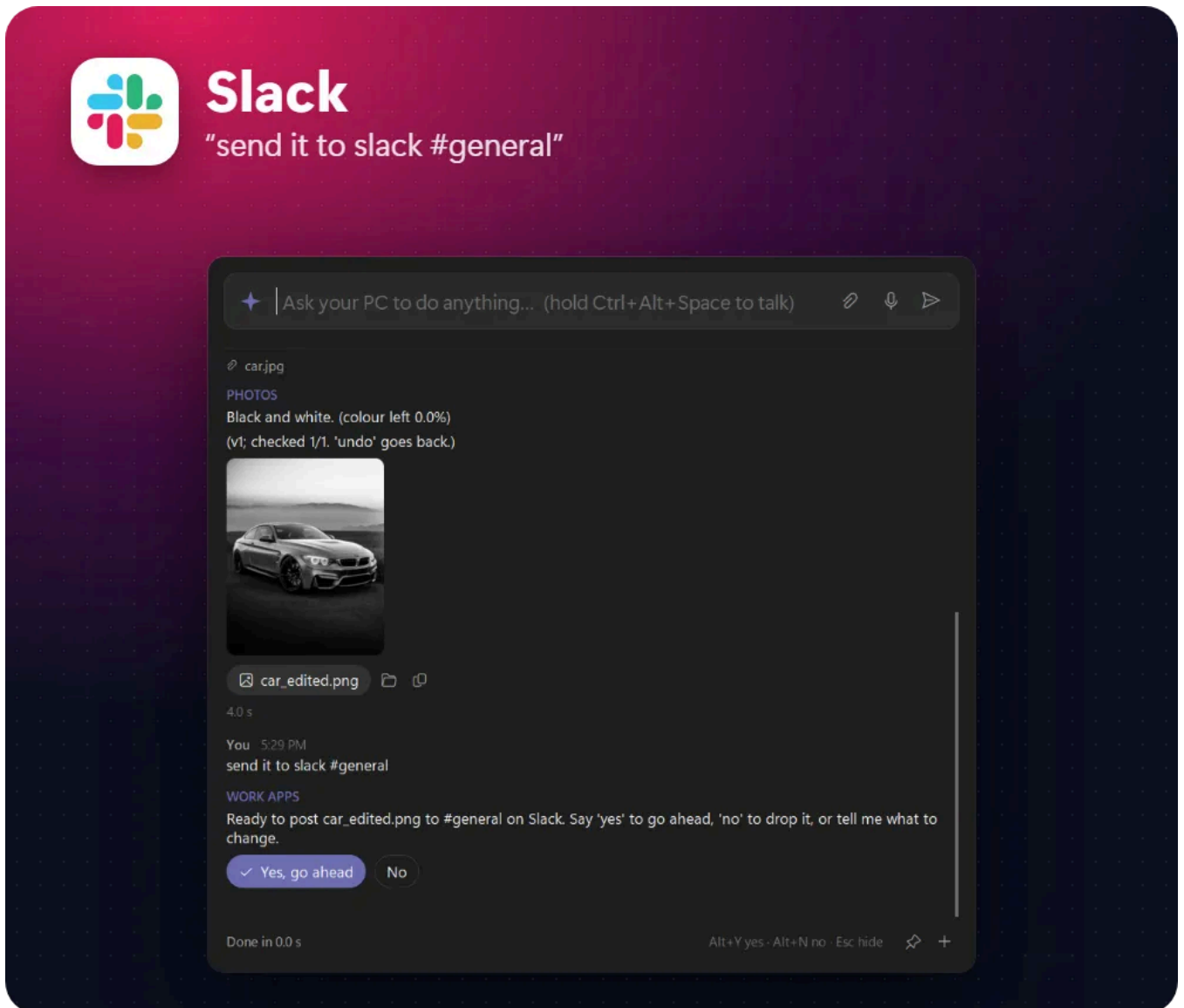
[Work apps] Uploaded v0_front.png to #team (checked). <link to the file>
total 38.7 s of work, 3 model calls, 3 tool calls, 0 screenshots, 12,645 input

Called one tool at a time, the same sentence takes five model calls: one before each tool, one to show the preview and one after the yes. The chain needs three.

A wrong upload can be deleted with undo, but a wrong email cannot be unsent. So the email tool only ever makes a Gmail draft, and sending the draft takes a second yes.



Nothing leaves the PC without a yes: preview, wait, three API calls, Slack's answer as the check, and undo (Created by Fareed Khan)



The same gate in the command bar after a black and white photo edit, waiting for Yes or No (Created by [Fareed Khan](#))

```
ledger.add("chain: house, 3D view, Slack post", 3, 3, 0, "12,645", "38.7 s", "30
```

Frame-Accurate Video Pipeline

Video editors are among the hardest programs for a screen agent. A timeline is dense, every drag has to land on the right frame, and the result only shows up after an export. They also happen to have some of the best native interfaces.

Timeline files on the frame grid

Premiere Pro, DaVinci Resolve and Final Cut Pro all import timeline files, and a file can put every cut exactly on a frame. The timeline tool takes clips, in and out points

and music, and writes four formats from one plan, with the frame rate snapped to a standard one and all times in whole frames.

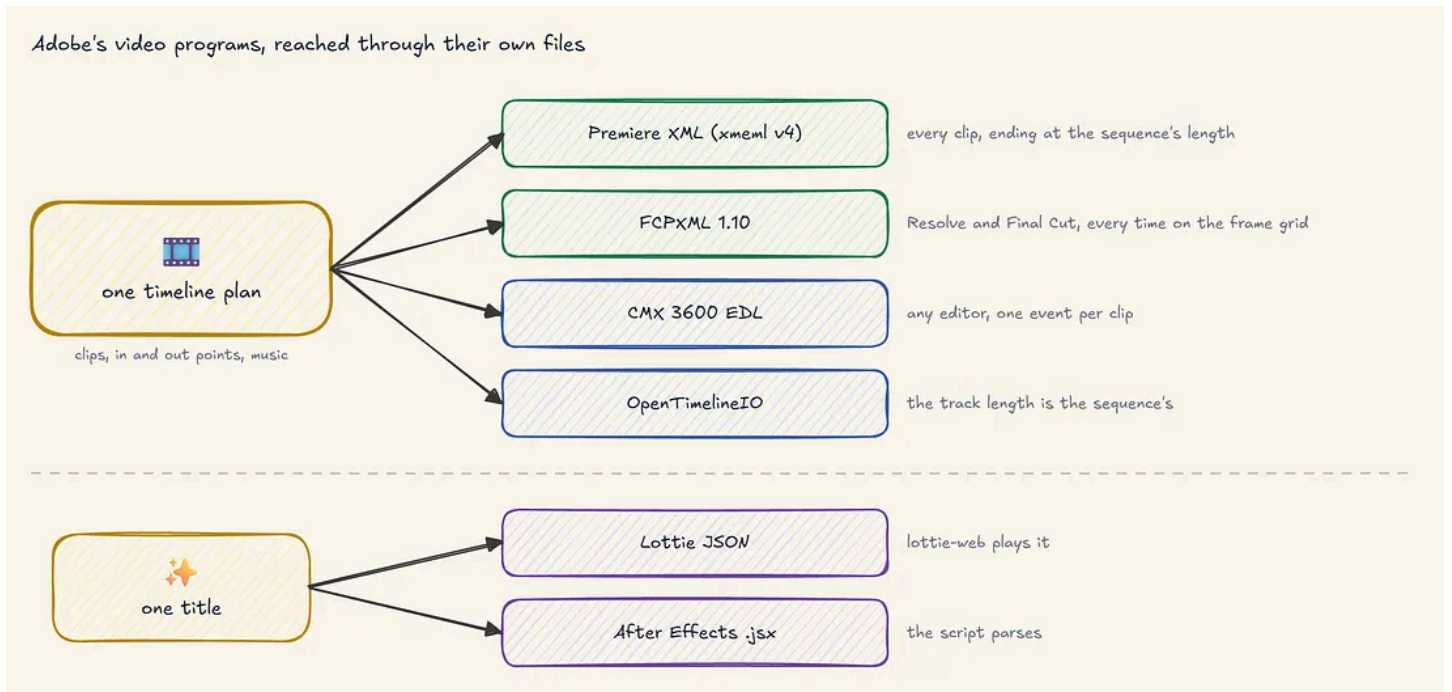
```
# apps/premiere.py
@tool()
def build_timeline(name: str, clips: list, music: str = "", fps: float = 30) ->
    """Write one timeline for Premiere Pro (XML), DaVinci Resolve and Final Cut
    OTIO. clips: [{"file": "in": seconds, "out": seconds}] in order. Every time
    fps = snap_fps(fps) # 29.97 stays 30000/1001
    tl = layout(clips, music, fps) # every cut in whole frames
    files = {fmt: write(fmt, tl, name) for fmt in ("premiere", "fcpxml", "edl",
    return result(summary(tl, files), files=files, checks=[read_back(fmt, f, tl)
```

Here is what the tool writes for the request in our integration test.

```
#### OUTPUT ####
> premiere timeline: clip1.mp4 1-4, clip2.mp4; music song.mp3; called 'Shop promo
model -> build_timeline(name="Shop promo", clips=[{"file": "clip1.mp4", "in": 1
            music="song.mp3")
timeline 3 s of clip1 + 8 s of clip2 = 330 frames at 30 fps, 1280x720
FCPXML clip1 offset="0s" start="1s" duration="3s"
        clip2 offset="3s" start="0s" duration="8s"
        music offset="1s" duration="11s" lane="-1" audioRole="music"
EDL 001 AX V C 00:00:01:00 00:00:04:00 00:00:00:00 00:00:00:00
Checked: Premiere XML: every clip, ending at the sequence's length; FCPXML: every clip, ending at the sequence's length; EDL: one event per clip; OTIO: the video track's length is the sequence's length
```

Each check reads its own file back. The FCPXML one divides every time by the frame duration and fails unless the answer is a whole number.

After Effects gets a `make_title_animation` tool. It writes the title as a Lottie animation, the JSON format After Effects exports through Bodymovin, and as an ExtendScript `.jsx` that builds the same composition. Headless Chrome plays the Lottie to check that the title is missing on the first frame and present on the last.



Adobe's video programs through their own files: one timeline in four formats, one title as a Lottie and a script (Created by [Fareed Khan](#))

```
ledger.add("timeline, 2 clips in 4 formats", 2, 1, 0, "2,530", "3.3 s", "4/4")
ledger.add("title animation, Lottie + .jsx", 2, 1, 0, "2,380", "6.1 s", "3/3")
```

One design call for a whole edit

CapCut, the program our screen agent failed on, has a native interface after all: its projects are draft files, which a tool can write straight into its project list. Only the export needs the screen. For that, the edit tool uses JianYing 5.9, CapCut's Chinese edition, with its windows parked past the right edge of every monitor.

The model works with two tools here. `analyze_clips` returns what is in each clip and a beat grid, and `cut_edit` takes a whole edit as its argument. The description of `cut_edit` starts with one rule.

```
# video/director.py: the rule at the top of cut_edit's description
DESIGN_SYSTEM = """You are a senior short-form and YouTube editor. You DESIGN t
writes every keyframe, so you choose and never compute. Output ONE JSON object:
```

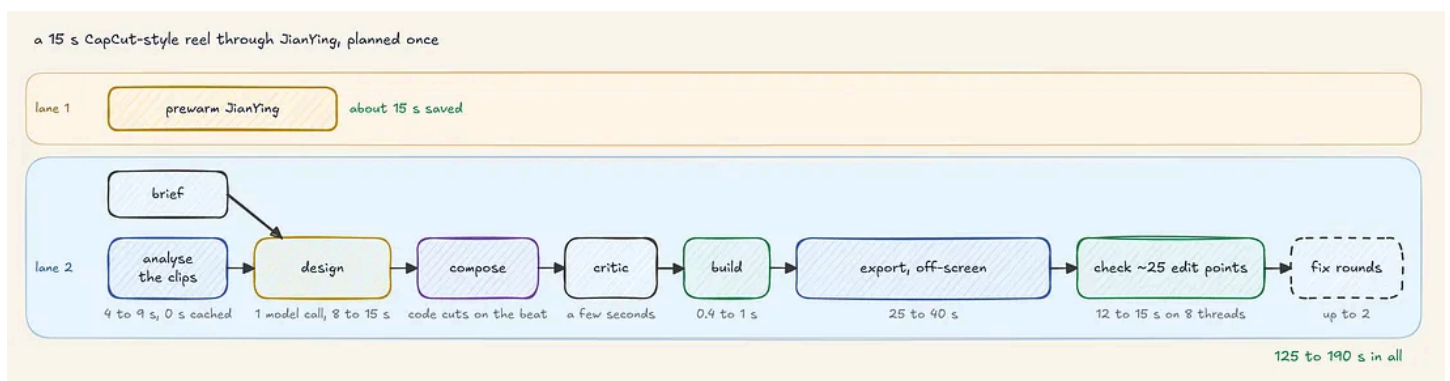
This design comes from my offline video tests: a hype edit on a 140 BPM grid with a speed ramp, a green-screen shot, a freeze-frame ending and a title that slams in at

the drop.

```
{
  "style": "hype",
  "canvas": "9:16",
  "music": {"generate": "hype", "bpm": 140},
  "shots": [
    {"id": "a", "section": "intro", "file": "mixkit_4832.mp4", "beats": 4},
    {"id": "b", "section": "drop", "file": "mixkit_47518.mp4", "beats": 6, "speed": 1.2},
    {"id": "c", "section": "drop", "file": "mixkit_28287.mp4", "beats": 2, "chrome": 1.5},
    {"id": "d", "section": "outro", "file": "mixkit_28287.mp4", "beats": 4, "speed": 0.8},
  ],
  "recipes": [{"use": "kinetic_title", "text": "BOSS MODE", "at": "drop"}]
}
```

Code turns “velocity” into a three-point speed ramp, each shot into whole beats and the title into keyframes. A more complex edit means a longer argument, while the number of model calls stays the same.

A render costs about 30 seconds. Before paying for one, `cut_edit` has a critic review the plan. About seventeen rule checks run first, such as “nothing happens in the first 1.5 s (no hook)”. One model review then proposes at most six patch operations, and a patch survives only if the weighted rule score does not get worse.



The edit tools: clip analysis, one design call, a critic, an off-screen export, and every edit point checked (Created by [Fareed Khan](#))

The rest runs in parallel and out of sight. JianYing starts while the model writes the design, which saves about 15 seconds. After the export, the edit points are checked on 8 threads. This is a run of the 15-second reel built on that design.

```
#### OUTPUT ####
model  1.6 s  in 1,520 tok -> analyze_clips(files=[3 clips]) (start
tool   7.4 s                                     <- analysed 3 file(s), 0 from cache, 31 candidates,
model 11.6 s  in 5,412 tok -> cut_edit(design={hype, 4 shots, BOSS MODE at the
tool                                     <- composed hype: 7 shots, drop at 6.9 s, 9 recipe e
                                     critic 2.6 s: score 7/10, 1 check(s) flagged; app
                                     built agent_boss_101512: 15.1 s, 9 tracks, 64 op
                                     exported agent_boss_101512 in 31.4 s
                                     verified 24 edit points in 13.2 s: 24 pass, 0 wa
                                     asked vs delivered: a hype reel from these 3 cli
model  1.0 s  in 7,180 tok -> reply
```

In my runs, a reel like this passed 24 of 24 edit points at about 100 seconds and \$0.008. A ten-genre batch of one-minute videos averaged about 3.3 minutes and \$0.009 per video, with 57 asks met, 21 partly met and none missed.

```
ledger.add("video edit, 15 s reel", "4 + vision", 2, 0, "21,400", "about 100 s"
```

The four model calls are the analysis request, the design, the critic and the reply. The vision calls look at each clip once and at each effect after the export.

Accessibility Tree Driver

Some programs have no file format, command line or API. Most Windows programs, though, describe their windows through UI Automation, a tree of controls with roles, names and actions that the model can read as text. We expose that tree as tools:

`observe_window` reads it, and `act` runs up to six actions on named controls.

```
# desktop/uia.py: what observe_window reads, in one cross-process call of about
def snapshot(hwnd, max_elements=300):
    arr = UIA.ElementFromHandle(hwnd).FindAllBuildCache(4, _COND_VISIBLE, _CR)
    els = []
    for i in range(min(arr.Length, max_elements * 4)):      # their properties a
        el = arr.GetElement(i)
        name, aid = el.CachedName or "", el.CachedAutomationId or ""
        ops = [op for op, prop in (("invoke", P_INVOKE), ("toggle", P_TOGGLE),
        if name or aid:
            els.append(El(f"e{len(els) + 1}", ROLES.get(el.CachedControlType, "
    return els
```

The model reads that tree as text, first for Calculator and then for CapCut.

```
#### OUTPUT ####
```

```
APP: calculatorapp.exe (pid 4242) | window "Calculator" | richness: rich (26 in
e3 Text "Display is 0" [CalculatorResults]
e13 Button "Seven" [num7Button] {invoke}
e28 Button "Equals" [equalButton] {invoke}
... (23 more)
```

```
APP: capcut.exe (pid 9120) | window "CapCut" | richness: EMPTY (0 interactable c
NOTE: the UI tree is thin; for anything not listed use ground_click with a plain
```

Calculator hands over every button by name. Because CapCut draws its own controls, UI Automation sees an empty window there, which is why the old agent had to work from screenshots.

what the planner reads at each step

 a screenshot



2,300 to 3,200 input tokens per call in our run

 the UI Automation tree

```
APP: calculatorapp.exe (pid 4242) | richness: rich (26 interactable of 31 elements)
e3 Text "Display is 0" [CalculatorResults]
e13 Button "Seven" [num7Button] {invoke}
e24 Button "Plus" [plusButton] {invoke}
e28 Button "Equals" [equalButton] {invoke}
```

a few hundred tokens of text, read in one call of about 50 to 100 ms

A screenshot costs the model 2,300 to 3,200 tokens a step, the UI Automation tree a few hundred tokens of text
(Created by [Fareed Khan](#))

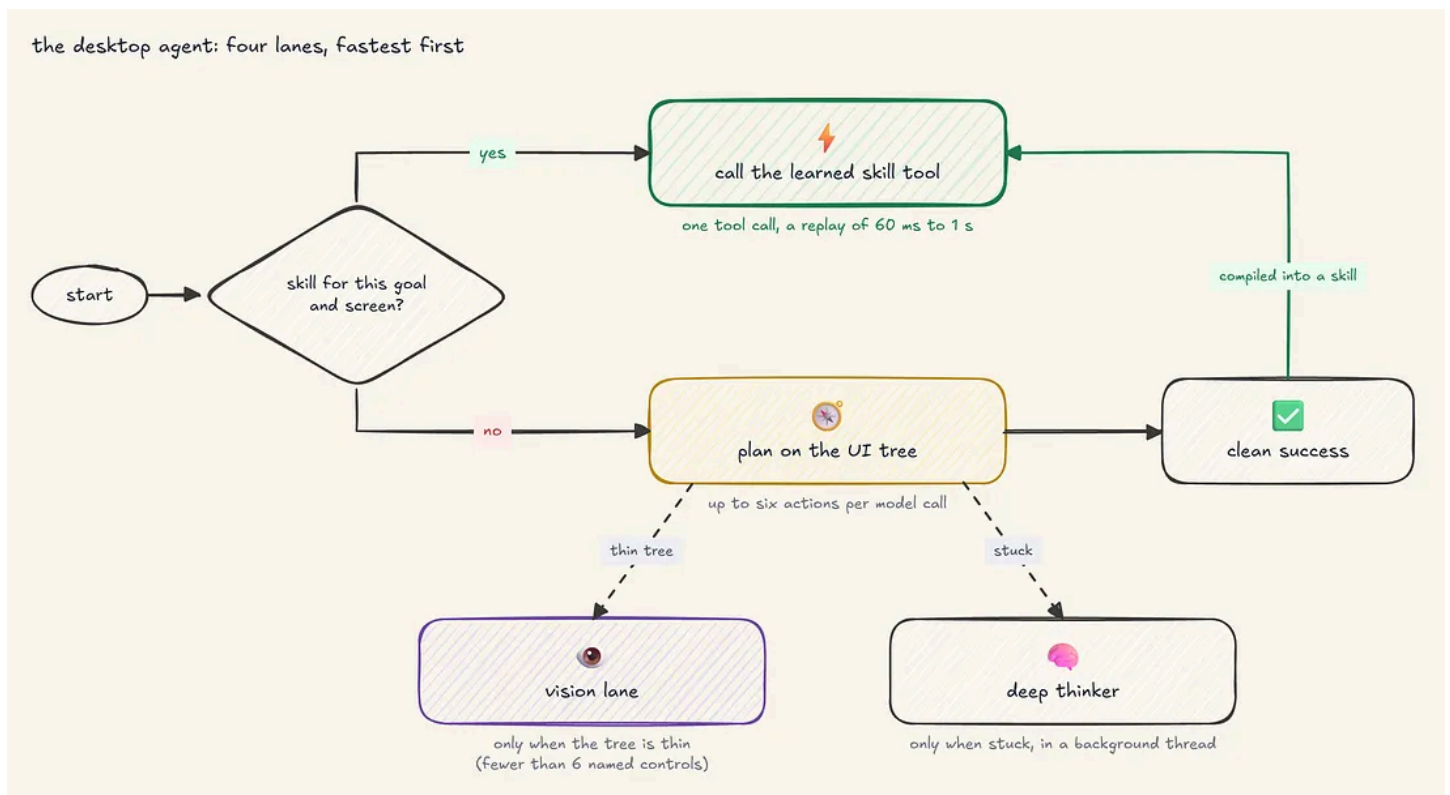
The tool's description asks for the cheapest reliable path, so the model types a whole expression in one action instead of pressing six buttons. When it finishes, it must quote the on-screen text that proves the answer.

```
{"tool": "act", "args": {"actions": [{"op": "type", "text": "25*4"},
                                     {"op": "key", "keys": ["enter"], "expect":
```

A few more habits keep it quick.

- **The program's own shortcuts.** CapCut ships about 80 editor shortcuts as a data file, such as `importMedia=Ctrl+I`, and the model is told they are “exact and instant, so prefer them to clicking”.
- **Pixels only when needed.** The `ground_click` tool, which finds a target in a screenshot, is offered only when the tree has fewer than 6 named, actionable controls.
- **No fixed sleeps.** `act` waits until the window has changed and held still, usually within 250 ms.
- **A deep thinker in the background.** A slower model plans hard tasks in a background thread while the quick planner starts on the obvious steps.

With the tree as its eyes, a new Calculator task took 3 model calls, 2 tool calls and 3.6 seconds. When the vision fallback does run, a small click model finds the spot: in my tests, Vocacla-2-500M found 29 of 30 CapCut targets.



The desktop agent's four lanes: a learned skill tool, planning on the UI tree, vision for thin trees, a deep thinker when stuck (Created by [Fareed Khan](#))

Every action also passes a safety check written in code, because text on the screen can carry instructions from an attacker. If a fake banner orders the model to press

“Delete all files”, the classifier still rates that click high risk, and an unattended run refuses it.

```
#### OUTPUT ####
invoke "Seven"                low      runs
key alt+f4                    medium   hotkey alt+f4 may close or delete some
invoke "Delete all files"     high     'Delete all files deleteAllButton' loc
type into a password field    block    refusing to type into a password field
key win+r                     block     system hotkey win+r is not allowed
```

```
ledger.add("window, new Calculator task", 3, 2, 2, "4,720", "3.6 s", "read back")
```

Skill compiler

That still leaves the fifth cost: a screen agent explores the program again every time it runs. Our agent compiles a clean run into a skill, with locators that name each control instead of its pixels and values turned into typed placeholders. The skill is registered as a new tool, and from then on the task takes the model one call. A skill runs only when the window looks like the one it learned on, and two failures in a row switch it off.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \geq 0.8$$

A skill replays only when the controls on screen now overlap the ones it was learned on by at least 80%

A skill replays only when today’s controls overlap the learned ones by at least 80% (Created by [Fareed Khan](#))

This is the skill compiled from one successful run of “calculate 12 + 30”, registered as a tool, and then called.

```
#### OUTPUT ####
compiled calculate-a-b-29da8f  intent 'calculate {a} + {b}'  params a, b (num
steps      [{"op": "type", "text": "{{a}}+{{b}}"}, {"op": "key", "keys": ["enter"]
final      read CalculatorResults (it showed "Display is 42")
tool       calculate_a_plus_b(a: number, b: number)
```

```

> calculate 25 + 4
model 1.2 s in 1,318 tok -> calculate_a_plus_b(a=25, b=4)
tool 0.05 s <- ok, read back "Display is 29"
model 0.8 s in 1,402 tok -> reply
25 + 4 = 29, read from Calculator's display.
total 2.1 s, 2 model calls, 1 tool call, 2,720 input tokens

'calculate twelve + thirty' -> no match: the placeholders only accept numbers

```

A six-press Calculator skill replays in **56 to 64 milliseconds**, and a Character Map task learned in 5.4 seconds replays in 1.0 second, app start included. OSExpert found the same lever: computer-use agents take 5 to 50 times longer than human experts, and learning skills from exploration closes about 80% of that gap.

```
ledger.add("window, learned skill as a tool", 2, 1, 2, "2,720", "2.1 s", "read l
```

Evaluation Against the Baseline

Now we print the full ledger, all thirteen rows, and check the four predictions against it.

```
ledger.show()
```

```

#### OUTPUT ####
task                                model tools  UI   tokens  seconds
CapCut filter, screen agent        13      0   32  30k-42k  155 s
layered image, Eid Sale poster      2       1    0   3,115   2.9 s
spreadsheet, marks sheet            2       1    0   6,915  18.9 s
slides, 10-slide deck               3       1    0   8,240   22 s
drafting, 10 marla house plan       2       1    0   3,268   4.4 s
3D views of the plan                2       1    0   3,410  33.8 s
building model, IFC4                2       1    0   3,190   4.6 s
chain: house, 3D view, Slack post   3       3    0  12,645  38.7 s
timeline, 2 clips in 4 formats      2       1    0   2,530   3.3 s
title animation, Lottie + .jsx      2       1    0   2,380   6.1 s
video edit, 15 s reel               4 + vision 2    0  21,400  about 100 s
window, new Calculator task          3       2    2   4,720   3.6 s
window, learned skill as a tool     2       1    2   2,720   2.1 s

```

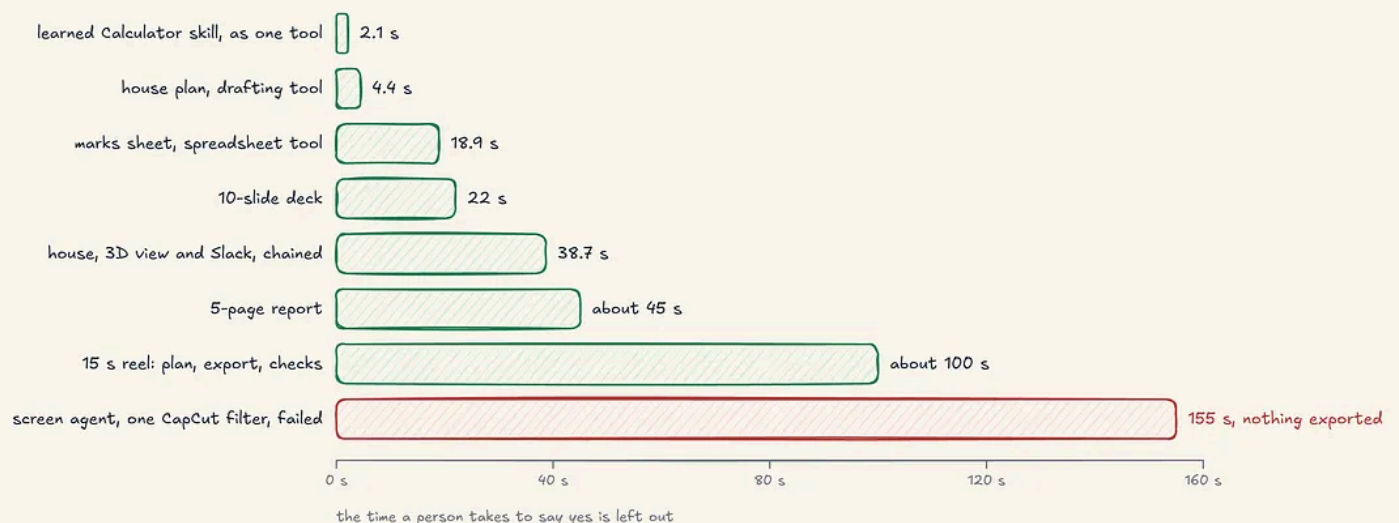
1. **Model calls fell from 13, for a failure, to 2 or 3 per task.** The video edit makes more, because its creative decisions need the model twice.
2. **Input tokens were 7 to 15 times lower for most rows,** about 2,400 to 4,700 against 30,000 to 42,000. This prediction holds only in part: the spreadsheet and the deck, with their long plans, were 4 to 5 times lower, and the video edit looks at pictures on purpose.
3. **Every row carries checks from inside its tool,** and all of them passed. A failed check would have stopped the chain at the step that caused it.
4. **Only the two window rows touched a visible program,** and they did it through the accessibility tree.

famous programs as tools: each through its native interface, each with a check

Photoshop layered PSD Pillow reads every layer	Illustrator layered SVG + PDF every layer named	Lightroom .xmp preset came out warmer	Premiere XML, FCPXML, EDL, OTIO every time on the frame grid
After Effects Lottie + .jsx lottie-web plays it	DaVinci Resolve FCPXML imports the timeline	Excel COM totals equal Python's	PowerPoint COM no text runs out of its box
AutoCAD DXF audits clean, 17/17	Blender bpy -b a floor in every room	Revit IFC4 the schema validator finds nothing	Solidworks STEP via FreeCAD same volume read back
MATLAB .m via Octave numbers re-checked in Python	Unity C# + editor script scripts and settings checked	KiCad kicad-cli ERC clean, netlist as designed	Slack Web API the same file id

Famous programs as tools: each through its native interface, each with its own check (Created by [Fareed Khan](#))

end to end, with every model call included



```
# ablate.py: the cost of removing each design choice, from unit costs measured earlier
shot, result_json = (1100, 2500), 260      # tokens: one 1080p screenshot, the cost of a
schema, tools, shortlist = 180, 200, 19    # tokens per schema, tools in the agent's
print(f"results as pictures          +{shot[0] - result_json[0]:,} to {shot[1] - result_json[1]:,} tokens per result")
print(f"every schema in the prompt +{schema * (tools - shortlist):,} tokens per schema")
print(f"one round trip per tool      +{(3 + 1) - 2} model calls for the three-tool round trip")
print(f"a cold Office per render    +{6 - 2} to {8 - 1} s per render")
```

To see which part of the design does the work, I removed one choice at a time and priced each removal with the unit costs measured earlier.

Beyond the ledger, the same pattern covers many more programs. Illustrator gets a layered SVG and a print PDF, Lightroom an .xmp preset checked on the photos it changes, SolidWorks STEP files that FreeCAD builds headless, and MATLAB scripts that run in GNU Octave with every printed number re-checked in Python. My test suite runs over 40 of these programs for real and passes 180 of 180.

```
#### OUTPUT ####
results as pictures          +840 to 2,240 tokens per result
every schema in the prompt +32,580 tokens per model call
one round trip per tool      +2 model calls for the three-tool sentence
a cold Office per render    +4 to 7 s per render
```

The shortlist saves the most tokens, with text results second. The chain saves only two calls, but each one is 1 to 2 seconds of a person waiting. The verifier is missing from the printout because removing it costs no time or tokens. Without it, a plan where Bed Room 1 loses its door would go out with no warning.

Known Limits

- **Every feature costs work up front.** Because each tool needs an implementation and checks, the agent covers about a hundred programs and services, not every program on Windows.
- **The model can still pick the wrong tool or the wrong arguments.** The shortlist can leave the right tool out, and a tool can only refuse what its checks can see.

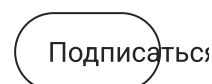
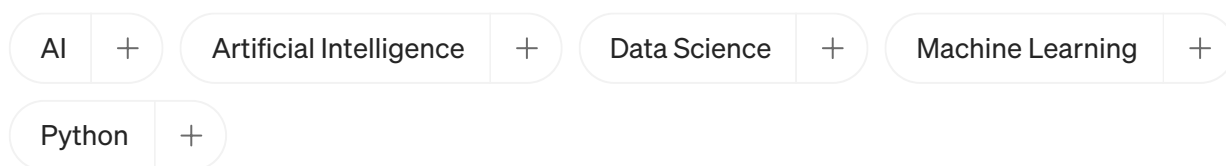
- **Checks prove numbers, not meaning.** The recorded marks-sheet plan has two slips no check catches: “Students Passed” counts grade F, and Percentage was typed as a percent, so 86 displays as 8600.0%.
- **Vision is the weak fallback.** Icon-only buttons are still hard to find.
- **Model calls still cost seconds.** Each one takes 1 to 3 seconds, partly network distance, so a task with many decisions stays bound by the model.
- **Budgets and platform.** A desktop run stops at 30 steps, 14 model calls or 180 seconds, and everything runs on Windows only.

Design Rules That Carry Over

My CapCut agent was slow because of the interface it was given, and a faster model would have shrunk only two of the six costs. **The fix was interface lifting:** giving the model each program’s features as typed tools, through the interface each program already has.

If I started another agent today, I would keep three habits from this one. I would look for each program’s own interface before reaching for a screenshot, wrap each feature as a typed tool that checks its own output, and hold anything other people will see until a person says yes.

The full code, with every program this post had no room for, is on [GitHub](#). Thanks for reading this far.



Автор: Фарид Хан

77 ТЫСЯЧ подписчиков · 0 подписчиков

Я пишу об искусственном интеллекте, <https://www.linkedin.com/in/fareed-khan-dev/>



Подписаться

Опубликовано в Level Up Coding

364 тыс. подписчиков · Опубликовано 1 час назад

Обучающие материалы и новости о программировании. Домашняя страница разработчика gitconnected.com && skilled.dev && levelup.dev
